Drupal 11
Zero to Hero

Interview-Ready Study Guide

Concepts · Navigation paths · Commands · Code · Exercises with answers · Interview Q&A · Cheat sheet

Written for Drupal 11.x (project version 11.4.8), PHP 8.3, Drush 13, DDEV. Built for a learner who knows HTML/CSS.

How to use this guide

Every chapter has the same pattern, so you can revise fast:

1. **What it is** (the meaning, in plain words, and the interview one-liner)
2. **How to do it** (admin navigation path or code)
3. **Commands / cheat codes** (and when to use them)
4. **Exercise + Answer** (do the exercise first, then check the answer)
5. **Interview questions** (collected in the last chapters too)

> **Two-day schedule.** Day 1: Chapters 1-9 (basics, site building, no code). Day 2: Chapters 10-17 (theming, modules, APIs, security, performance, deployment). Chapters 18-20 (cheat sheet, interview Q&A, checklist) are for the night before the interview.

> **Honest note.** Two days builds a strong foundation and interview vocabulary. Interviewers also test hands-on experience, so build every exercise yourself, do not only read the answers.

Contents

1. Drupal basics and architecture
2. Your environment: DDEV, Composer, Drush
3. Admin navigation map (Drupal 11)
4. Content types, fields, view modes
5. Views
6. Taxonomy, menus, blocks, media
7. Users, roles, permissions
8. Modules, themes, configuration management, recipes
9. Popular contributed modules and site-building tools
10. Theming with Twig
11. Custom module development (routes, controllers, blocks, forms)
12. Services, dependency injection, hooks, events, plugins
13. Entity API, database API, config API, state, cache, queue and cron
14. Security
15. Performance and caching
16. Testing, coding standards, debugging
17. Migrate, JSON:API, headless, multilingual, deployment
18. Drush and Composer cheat sheet
19. Interview Q&A (100+ questions)
20. Final revision checklist and mini-project

1. Drupal basics and architecture

1.1 What is Drupal?

Definition: Drupal is a free, open-source **content management framework (CMF)** written in **PHP**, built on **Symfony** components, using a database (MySQL/MariaDB/PostgreSQL/SQLite), and managed with **Composer**.

> **Interview one-liner:** "Drupal is an open-source, API-first content management framework on PHP and Symfony. It is strongest for complex content models, multilingual sites, granular permissions and large enterprise or government sites."

1.2 CMS vs CMF vs framework

Term	Meaning
CMS	Software to create and manage content through a UI (WordPress, Drupal).
CMF	Content management framework: a CMS whose pieces you can extend and reuse as a framework. Drupal fits here.
Framework	Code library for building apps (Symfony, Laravel). Drupal uses Symfony underneath.

1.3 Core terminology (know every word)

Term	Meaning
Core	The code that ships with Drupal (`web/core`). Never edit it.
Contrib	Modules and themes from drupal.org, in `web/modules/contrib` and `web/themes/contrib` . Installed with Composer.
Custom	Your own code in `web/modules/custom` and `web/themes/custom` .
Node	A piece of content (an article, a page).
Content type	A template (bundle) for nodes: Article, Page, Event.
Entity	Any data object Drupal manages: node, user, term, file, media, comment, block content, config entity.
Bundle	A sub-type of an entity type. Node is the entity type, Article is a bundle.
Field	A piece of data on an entity (text, date, image, reference).
Widget	The input control for a field on the edit form.
Formatter	How a field is displayed on output.
View mode	A named way to display an entity (Full, Teaser, Card).

Block	A reusable piece of output placed in a theme region.
Region	An area defined by a theme (header, sidebar, footer).
Theme	The presentation layer: Twig templates, CSS, JS.
Module	Code that adds functionality.
Hook	A function (or, in Drupal 11, a class method with the `#[Hook]` attribute) that lets a module alter or react to core behavior.
Plugin	A swappable piece of functionality discovered automatically (blocks, field types, widgets, formatters, views handlers, queue workers).
Service	A reusable PHP object managed by the service container (database, logger, current user).
Dependency injection (DI)	Passing a class its dependencies through the constructor instead of creating them inside it.
Render array	A PHP array that describes output; Drupal turns it into HTML.
Twig	The template engine used for themes.
Configuration	Site settings stored in YAML (content types, views, blocks). Exportable.
Content	User data stored in the database (nodes, users). Not in config export.
Recipe	A reusable package of config, content and module installs applied with `drush recipe` or `php core/scripts/drupal recipe`.

1.4 Configuration vs content (very common interview question)

	Configuration	**Content**
Examples	Content types, fields, views, roles, blocks, menus config	Nodes, users, comments, taxonomy terms, media
Stored	DB, but exportable to YAML in `config/sync`	DB only
Moves between environments by	`drush cex` / `drush cim`	Migrations, `default_content`, or DB copy

1.5 How a Drupal request works (explain this in interviews)

1. The browser requests `/events`; the web server sends it to `web/index.php`.
2. Drupal boots the **kernel** (Symfony HttpKernel) and the **service container**.
3. **Routing** matches the path to a route (from a `*.routing.yml` file or a View).
4. **Access checks** decide whether the user may view it.
5. A **controller** (or View, or Entity handler) returns a **render array**.
6. The **render system** converts it to HTML using **Twig templates** and the theme, applying **cache metadata**.
7. A **Response** is returned to the browser.

1.6 Folder structure of your project

```
drupal112-training/
  .ddev/               DDEV config (docker environment)
  composer.json        PHP dependencies (Drupal itself, modules)
  composer.lock        exact versions installed
  vendor/              PHP libraries (Symfony, Drush...) - never edit
  recipes/             Drupal recipes
  config/sync/         exported configuration (created by you)
  web/                 the public document root
    core/              Drupal core - never edit
    modules/contrib/   downloaded modules
    modules/custom/    your modules
    themes/contrib/    downloaded themes
    themes/custom/     your themes
    profiles/          install profiles
    sites/default/
       settings.php        main settings (DB, config sync path, salt)
       settings.ddev.php   DDEV generated DB settings
       files/              uploaded files (public://)
    index.php          front controller
```

1.7 Release cycle (interview favourite)

- Drupal uses **semantic versioning**: 11.x. A new minor version (11.1, 11.2...) comes about every six months; patch releases fix bugs and security.
- Major versions (10, 11, 12) remove deprecated code. Move from 10 to 11 by updating to the latest 10.x, fixing deprecations (tool: **Upgrade Status** module, **drupal-check**, **Rector**) and then running `composer require drupal/core-recommended:^11 ...`.
- **Drupal CMS** is a product built on Drupal core with recipes and a friendly editor experience. It is not a different codebase.

1.8 Drupal vs WordPress (asked often)

Topic	Drupal	WordPress
Content modelling	Custom entities, fields, references in core	Posts/pages; custom types via plugins
Permissions	Granular roles and permissions in core	Basic roles; plugins for more
Multilingual	Built in core	Plugin required
Learning curve	Steeper	Gentler
Best fit	Complex, enterprise, government, multi-site	Blogs, small business sites
Dependency mgmt	Composer	Plugin directory

Exercise 1.1

> **Task:** Open your project folder and identify: where core lives, where you will put a custom module, where uploaded files go, and where config will be exported.

> **Answer:** Core: `web/core`. Custom module: `web/modules/custom/<name>`. Uploaded files: `web/sites/default/files`. Config export: the path in `$settings['config_sync_directory']` in `settings.php` (we will use `../config/sync`).

2. Your environment: DDEV, Composer, Drush

2.1 What each tool is

Tool	Meaning	When you use it
DDEV	Docker-based local dev environment (web server, PHP, database)	Start and stop the site, run commands inside the container
Composer	PHP dependency manager	Install/update/remove Drupal, modules, themes, libraries
Drush	Drupal command-line tool	Clear cache, export config, run updates, create users, install modules

2.2 DDEV commands

Command	What it does
`ddev start`	Start the project containers
`ddev stop`	Stop containers
`ddev restart`	Restart after changing config
`ddev describe`	Show URLs, DB credentials, ports
`ddev launch`	Open the site in the browser
`ddev ssh`	Shell inside the web container
`ddev drush ...`	Run Drush inside the container
`ddev composer ...`	Run Composer inside the container
`ddev import-db --file=db.sql.gz`	Import a database dump
`ddev export-db --file=db.sql.gz`	Export the database
`ddev logs`	View web server logs
`ddev xdebug on`	Enable step debugging
`ddev mailpit`	Open the mail catcher (see outgoing email)
`ddev snapshot`	Save a database snapshot (backup before risky steps)

2.3 Installing a fresh Drupal 11 site with DDEV (to explain in interviews)

```
mkdir mysite && cd mysite
ddev config --project-type=drupal11 --docroot=web
ddev start
ddev composer create-project drupal/recommended-project:^11 .
ddev composer require drush/drush
ddev drush site:install standard --account-name=admin --account-pass=admin -y
ddev drush uli        # one-time login link
ddev launch
```

2.4 Composer essentials

Command	Meaning / use
`ddev composer require drupal/pathauto`	Download a contrib module and add it to composer.json
`ddev composer require drupal/pathauto:^1.13`	Specific version constraint
`ddev composer remove drupal/pathauto`	Remove a package (uninstall the module in Drupal first!)
`ddev composer update drupal/core-recommended drupal/core-composer-scaffold --with-all-dependencies`	Update Drupal core
`ddev composer update drupal/pathauto --with-dependencies`	Update one module
`ddev composer outdated "drupal/*"`	List outdated Drupal packages
`ddev composer audit`	Check for known security advisories
`ddev composer install`	Install exactly what composer.lock says (used on servers and by teammates)
`ddev composer why-not drupal/core 11.5`	Find out why an upgrade is blocked

> **Meaning of version signs:** `^1.2` = any 1.x at or above 1.2 (not 2.0). `~1.2.3` = 1.2.x at or above 1.2.3. `*` = any. `dev-main` = branch.

> **composer.json vs composer.lock:** composer.json lists what you asked for; composer.lock records the exact versions installed. Commit both. On deployment run `composer install` (reads the lock), not `update`.

> **Never** download modules by hand into `web/modules` and **never** edit core or contrib. Use Composer, and for fixes use patches (`cweagans/composer-patches`).

2.5 Drush essentials (more in Chapter 18)

Command (alias)	When to use
`drush cr` (cache-rebuild)	Clear all caches. After changing Twig, services, routing, info.yml, libraries.

`drush uli`	One-time admin login link when you forgot the password
`drush en pathauto -y` (pm:install)	Install a module
`drush pmu pathauto -y` (pm:uninstall)	Uninstall a module
`drush cex -y` (config:export)	Export active config to YAML files
`drush cim -y` (config:import)	Import YAML config into the site
`drush updb -y` (updatedb)	Run database updates after code update
`drush status` (st)	Show Drupal version, DB, paths
`drush watchdog:show` (ws)	Show recent logs
`drush user:create editor1 --mail=e@x.com --password=pass`	Create a user
`drush user:role:add editor editor1`	Give a role to a user
`drush sql:dump > backup.sql`	Database backup
`drush php:eval "echo 1;"`	Run quick PHP
`drush generate module`	Scaffold code (modules, controllers, forms, plugins)

Exercise 2.1

> **Task:** (a) Start the project and log in as admin without knowing the password. (b) Install the Pathauto module. (c) Check your Drupal version.

Answer:

```
ddev start
ddev drush uli                      # open the link it prints
ddev composer require drupal/token drupal/pathauto
ddev drush en token pathauto -y
ddev drush status                   # shows Drupal version
```

3. Admin navigation map (Drupal 11)

Drupal 11 offers the classic **Toolbar** (top bar plus a drawer) and the newer **Navigation** module (a left vertical sidebar, in core). Both lead to the same pages. The **Gin** admin theme is the modern admin look. Learn the URL; it works no matter which menu you use.

Task	Menu path	URL
All content	Content	`/admin/content`
Add content	Content → Add content	`/node/add`
Custom block library	Content → Blocks	`/admin/content/block`
Media library	Content → Media	`/admin/content/media`
Files	Content → Files	`/admin/content/files`
Comments	Content → Comments	`/admin/content/comment`
Content types	Structure → Content types	`/admin/structure/types`
Fields of a type	Structure → Content types → Manage fields	`/admin/structure/types/manage/TYPE/fields`
Form display	… → Manage form display	`.../TYPE/form-display`
Output display	… → Manage display	`.../TYPE/display`
Views	Structure → Views	`/admin/structure/views`
Add view	Structure → Views → Add view	`/admin/structure/views/add`
Block layout	Structure → Block layout	`/admin/structure/block`
Menus	Structure → Menus	`/admin/structure/menu`
Taxonomy	Structure → Taxonomy	`/admin/structure/taxonomy`
Display modes	Structure → Display modes	`/admin/structure/display-modes`
Media types	Structure → Media types	`/admin/structure/media`
Block content types	Structure → Block content types	`/admin/structure/block-content`
URL aliases	Configuration → Search and metadata → URL aliases	`/admin/config/search/path`
Pathauto patterns	Configuration → Search and metadata → URL aliases → Patterns	`/admin/config/search/path/patterns`
Appearance (themes)	Appearance	`/admin/appearance`
Modules	Extend	`/admin/modules`
Uninstall modules	Extend → Uninstall	`/admin/modules/uninstall`

Site information	Configuration → System → Basic site settings	`/admin/config/system/site-information`
Text formats and editors	Configuration → Content authoring → Text formats	`/admin/config/content/formats`
Performance (caching)	Configuration → Development → Performance	`/admin/config/development/performance`
Logging	Configuration → Development → Logging and errors	`/admin/config/development/logging`
Languages	Configuration → Regional and language → Languages	`/admin/config/regional/language`
Regional settings (timezone)	Configuration → Regional and language → Regional settings	`/admin/config/regional/settings`
Config sync UI	Configuration → Development → Configuration synchronization	`/admin/config/development/configuration`
Cron	Configuration → System → Cron	`/admin/config/system/cron`
Image styles	Configuration → Media → Image styles	`/admin/config/media/image-styles`
People	People	`/admin/people`
Roles	People → Roles	`/admin/people/roles`
Permissions	People → Permissions	`/admin/people/permissions`
Account settings	Configuration → People → Account settings	`/admin/config/people/accounts`
Status report	Reports → Status report	`/admin/reports/status`
Recent log messages	Reports → Recent log messages	`/admin/reports/dblog`
Available updates	Reports → Available updates	`/admin/reports/updates`
Field list	Reports → Field list	`/admin/reports/fields`
Help	Help	`/admin/help`

> **Habit:** When something misbehaves, check `/admin/reports/dblog` and `/admin/reports/status` first, then run `ddev drush cr`.

4. Content types, fields, view modes

4.1 Meaning

Content type = a bundle of the node entity type with its own fields and settings. **Field** = typed data attached to an entity bundle. A field has a **storage** (type, cardinality: shared across bundles) and an **instance/config** (label, required, per bundle).

> **Interview one-liner:** "A content type defines the structure of a kind of content. Fields hold the data, widgets control the edit form, formatters control the output, and view modes let the same content render differently in teasers, cards or full pages."

4.2 Core field types

Field type	Use for
Text (plain / formatted / long / with summary)	Titles, descriptions, body
Number (integer, decimal, float)	Price, quantity
Boolean	Yes/no flags (Featured)
Date / Date range / Timestamp	Event dates
List (text, integer, float)	Fixed options (select, radios, checkboxes)
Link	URL with title
Email, Telephone	Contact info
Entity reference (Content, Taxonomy term, User, Media, File, Custom block)	Relationships between items
Image / File	Direct uploads (media is preferred for reuse)
Entity reference revisions	Used by Paragraphs (contrib)

4.3 How to create a content type (navigation)

1. **Structure** → **Content types** → **Add content type** (`/admin/structure/types/add`).
2. Name: Event. Description. In **Publishing options**: default published, create new revision. In **Display settings**: toggle "Display author and date information". In **Menu settings**: allowed menus. Save.
3. Click **Manage fields** → **Create a new field** → choose type → set label & machine name (`field_event_date`) → set storage settings (cannot change type later) → field settings (required, help text, default) → Save.
4. **Manage form display**: drag fields to reorder; choose widget (Date picker, Autocomplete, Select list, Check boxes/radios, Media library).

5. **Manage display**: choose formatter, label position (above, inline, hidden), and which fields appear in **Default**, **Teaser**, other view modes. Use "Custom display settings" at the bottom to enable more view modes for that bundle.

> **Gotcha:** The field machine name and type are permanent. Plan names before saving. Deleting a field deletes its data.

4.4 Entity reference settings

- **Reference type**: what is referenced (Content, Taxonomy term, User, Media).
- **Reference method**: Default (choose bundles, sort) or View (use a Views display to filter choices).
- **Create referenced entities if they don't already exist**: lets editors add new terms on the fly with autocomplete (tags).
- **Allowed number of values**: Limited (1, 2...), or Unlimited.

4.5 View modes and form modes

- **View mode**: output variant (full, teaser, search result, card). Create new ones at **Structure** → **Display modes** → **View modes** (`/admin/structure/display-modes/view`).
- **Form mode**: edit form variant. **Structure** → **Display modes** → **Form modes**.

Use a view mode when you need the same content shown differently in different places (a list card vs a full page). In a View, choose "Show: Content" with a view mode, or "Fields".

4.6 Revisions, moderation, status

- **Revisions**: every save can create a new version (the "Revisions" tab on a node). You can revert.
- **Content Moderation** (core module) and **Workflows** (core): draft → review → published states. Enable both modules, then go to **Configuration** → **Workflow** → **Workflows** (`/admin/config/workflow/workflows`), edit "Editorial", and attach it to the Event content type.
- **Published** flag: unpublished nodes are visible only to users with the right permission.

4.7 URL aliases

- Every node has a system path `/node/5` . An **alias** like `/events/my-concert` is nicer.
- Manual: node edit form → **URL alias** panel. Automatic: **Pathauto** module, pattern `/events/[node:title]` at `/admin/config/search/path/patterns` .

Exercise 4.1 — Event content type

> **Task:** Create content type **Event** with: Date range (`field_event_date`), Location (plain text, `field_location`), Featured (boolean, `field_featured`), Category (taxonomy reference to vocabulary "Event category", multiple values), Image (media reference). Create 3 events. Make the Teaser show only title and image. Add a Pathauto pattern `/events/[node:title]` .

Answer (steps):

1. Install pathauto (Exercise 2.1). Create vocabulary first: **Structure** → **Taxonomy** → **Add vocabulary** "Event category"; add terms Music, Tech, Sports.
2. **Structure** → **Content types** → **Add content type** "Event". Save and go to Manage fields.
3. Add fields: Date → "Date range" (`event_date`); Text (plain) → `location` ; Boolean → `featured` ; Reference → Taxonomy term → `category` (choose vocabulary, unlimited); Reference → Media → `image` (Image media type).
4. **Manage display** → scroll to "Custom display settings" → ensure Teaser is checked → open the Teaser tab → hide all except title (the title is shown by the node template) and Image (formatter "Rendered entity" or an image style). Save.
5. **Configuration** → **Search and metadata** → **URL aliases** → **Patterns** → **Add**: Pattern type "Content", pattern `/events/[node:title]` , select bundle Event.
6. **Content** → **Add content** → **Event** three times.

5. Views

5.1 Meaning

Views is a query builder with a UI (in core). It builds a SQL query against entities, applies filters and sorts, and renders the result as a page, block, feed, or other display.

> **Interview one-liner:** "Views lets site builders create lists and reports without writing SQL or PHP. You choose the base table, fields, filters, sorts, relationships and a display such as a page or block."

5.2 Views vocabulary

Term	Meaning
View	The saved configuration (a config entity).
Display	An output of the view: Page, Block, Feed, Attachment, Embed, REST export, Entity reference.
Format	The layout: Unformatted list, Table, Grid, HTML list, Responsive grid, Serializer.
Row style	Whether rows show Fields or rendered Content (entity in a view mode).
Fields	Columns of data.
Filter criteria	Conditions applied to the query (fixed or exposed to visitors).
Exposed filter	A filter shown as a form to the visitor.
Sort criteria	ORDER BY, can also be exposed.
Relationship	A JOIN to a related entity so you can use its fields (example: content → author).
Contextual filter	A filter whose value comes from the URL or context (argument). Example: `/events/music`.
Pager	Pagination (full, mini, "display a specified number", "display all").
Header / Footer / Empty	Text or areas above, below, and when there are no results.
Aggregation	GROUP BY with COUNT, SUM, etc.
Rewrite results	Change a field's output using tokens like `{{ title }}`.
Override	Setting applies to one display only, not "All displays".

5.3 Create a view (navigation)

1. **Structure** → **Views** → **Add view** (`/admin/structure/views/add`).
2. View name; Show: Content of type Event sorted by Newest first.

3. Tick **Create a page**: title, path `events`, display format (Table or Grid of fields), items per page.
4. Tick **Create a block** (optional): items 3.
5. **Save and edit**.

The editing screen

- Top: displays list. Left column: Title, Format, Fields, Filter, Sort. Right: Advanced (contextual filters, relationships, exposed form, query settings, caching, header, footer, no results).
- Click **Add** next to Fields / Filter criteria / Sort criteria, search by name, tick it, **Add and configure**.
- The dropdown **For: All displays / This page (override)** decides scope. Choose carefully.
- Bottom **Preview**, with "Auto preview" ticked, shows results as you edit.
- Always click **Save**.

5.4 Exposed filter example (Category)

1. Filter criteria → Add → "Event: Category (field_category)" → Add and configure.
2. Tick **Expose this filter to visitors**. Set "Filter type to expose": Dropdown. Label: Category. Tick "Limit list to selected items" optional.
3. Apply. Exposed form settings: Advanced → Exposed form → Exposed form style "Basic", change "Submit button text" to "Search".

5.5 Contextual filter example (category in URL)

1. Advanced → Contextual filters → Add → "Content: Has taxonomy term ID".
2. "When the filter value is NOT in the URL": Display all results / Hide view / Provide default value.
3. "When the filter value IS in the URL": validation criteria "Taxonomy term" so a bad ID shows "page not found".
4. Page path: `events/%`. Visiting `/events/3` filters by term 3.

5.6 Relationship example

Show event author name: Advanced → Relationships → Add → "Content author" → then add field "User: Name", choosing the relationship "author". Without a relationship, you can only use fields of the base entity.

5.7 Useful Views settings

- **Query settings** → **Disable SQL rewriting**: use rarely; it bypasses node access (security risk).
- **Caching**: Tag based (default); Time based for heavy views.
- **Access**: Permission or Role. Always set it on pages.
- **Use AJAX**: updates the list without reloading the page.
- **Show SQL query**: enable at `/admin/structure/views/settings` for debugging.
- **Distinct**: removes duplicates (caused by multi-value fields in joins).
- **Machine name / Path / Menu**: A page display can add itself to a menu (Menu: Normal menu entry).
- **Contextual links** and **Field rewriting** let you control markup.

5.8 Common Views mistakes (interview-worthy)

- Editing "All displays" when you meant only one display.
- Duplicate rows (multi-value fields, relationships) → fix with Distinct or aggregation.
- Forgetting Published = Yes filter, which shows unpublished content to editors in a public list.
- Forgetting that views cache: after a change that doesn't appear, check caching and clear caches.

Exercise 5.1 — Upcoming events view

Task: Create a view "Upcoming events": page at `/events` as a table (Title linked, Date, Location, Image), only published Events whose date is today or later, sorted by date ascending, an exposed Category filter, "No events found" text, and a block with the next 3 events. Make a second page `/events/%` filtering by category ID.

Answer (steps):

1. Add view → Show Content of type Event → sort "Unsorted" (you will add your own) → Create a page, path `events`, display format Table of fields, 10 items → Create a block, 3 items → Save and edit.
2. Fields: add Event date, Location, Image (Media), keep Title (check "Link to content").
3. Filter criteria: keep Published (=Yes) and Content type = Event. Add "Event date (field_event_date)" → operator "Is greater than or equal to" → value type "An offset from the current time" → value `now` (use the start field).
4. Sort criteria: add Event date ascending (use "For: All displays").
5. Expose Category filter (5.4) — For: **This page (override)** so the block stays simple.
6. Advanced → No results behavior → Add "Global: Text area" → "No events found."
7. Block display: Pager → "Display a specified number of items | 3" (This block override). Footer → Add Global Text area with a link to `/events`.
8. Contextual filter for term ID with path `events/%`: Duplicate the page display, change path to `events/%`, override contextual filters for this display only.
9. Save. Test.

6. Taxonomy, menus, blocks, media

6.1 Taxonomy

Meaning: a classification system. A **vocabulary** contains **terms**. Terms can be hierarchical.

- **Create:** Structure → Taxonomy → Add vocabulary → Add terms.
- **Hierarchy:** List terms tab → drag a term under another.
- **Use:** add a Reference → Taxonomy term field to a content type (widgets: autocomplete (tags style), select, check boxes/radios).
- Term pages at `/taxonomy/term/ID` come from the core view "Taxonomy term".
- **Tags vs categories:** tags are free-form (autocomplete with create-on-the-fly); categories are a controlled list.

6.2 Menus

- Core menus: Main navigation, Footer, Administration, Tools, User account.
- **Add a link:** Structure → Menus → Main navigation → Add link: Title, Link (`/events` or start typing a title), Enabled, Show as expanded, Parent, Weight.
- **From a node:** Node edit form → Menu settings → Provide a menu link.
- **Display:** a menu is only visible after its block is placed (6.3).
- Menu links can be defined in code with `*.links.menu.yml`.

6.3 Blocks and regions

- **Place a block:** Structure → Block layout → choose the theme tab → **Place block** next to a region → pick block → configure title, **Visibility** (pages, content types, roles, languages) → Save → Save blocks.
- **Custom block (block content):** Content → Blocks → Add content block (e.g. "Basic block" with Body). Reusable blocks can be placed multiple times. Non-reusable ones belong to a single place (Layout Builder).
- **Block types:** Structure → Block content types, to add fields to custom blocks.
- **Views blocks** appear in the block list as "Views: <view name>".
- **Layout Builder** (core): per-content-type or per-node drag-and-drop layouts. Enable `layout_builder` and `layout_discovery`. Then Manage display → tick "Use Layout Builder".

6.4 Media

- Enable **Media** and **Media Library** modules (default in Standard profile).
- **Media types:** Structure → Media types (Image, Document, Audio, Video, Remote video). Each is a bundle with source field and extra fields (alt, caption).
- **Add media:** Content → Media → Add media.
- **Use:** Add a "Media" reference field; widget "Media library".

- **Image styles:** Configuration → Media → Image styles. Define scale/crop/convert (e.g. 400x300 WebP) and select it in the formatter. Responsive image styles give srcset.

6.5 Text formats and CKEditor 5

- Configuration → Content authoring → Text formats and editors.
- **Basic HTML**, **Restricted HTML** (no editor, anonymous users), **Full HTML** (trusted roles only).
- Each format has an editor toolbar and an allowed-HTML filter. A security rule: never give anonymous users Full HTML.

Exercise 6.1

> **Task:** (1) Add "Events" to the main menu. (2) Make Tech a child of a "Topics" term. (3) Create a custom block "Welcome" and show it only on the front page, in the sidebar. (4) Create an image style "Event card" (scale and crop 400x300, WebP) and use it for the Event image in Teaser.

> **Answer:**
>
> 1. Structure → Menus → Main navigation → Add link, title Events, link `/events`, Save.
> 2. Structure → Taxonomy → Event category → List terms → drag Tech to the right under the parent term.
> 3. Content → Blocks → Add content block → Basic block, "Welcome" → Save. Structure → Block layout → Place block in Sidebar → Visibility → Pages → `<front>` → "Show for the listed pages".
> 4. Configuration → Media → Image styles → Add → "event_card" → Add effect "Scale and crop" 400x300, Add effect "Convert" webp → Create. Then Manage display → Teaser → Image field formatter → choose the style.

7. Users, roles, permissions

7.1 Meaning

- **User** = an account (an entity). **User 1** (uid 1) bypasses all access checks.
- **Role** = a set of permissions. Built-in: **Anonymous**, **Authenticated** (every logged-in user inherits it), and **Administrator**.
- **Permission** = a single allowed action, provided by modules (e.g. "Event: Create new content").

> **Interview one-liner:** "Permissions are granted to roles, and roles are assigned to users, never permissions directly to users. Anonymous and Authenticated are separate built-in roles: a logged-in user does not inherit the Anonymous role's permissions, only Authenticated plus any extra roles assigned."

7.2 Navigation

- **Add role:** People → Roles → Add role (`/admin/people/roles/add`).
- **Permissions grid:** People → Permissions (`/admin/people/permissions`) or per role (`/admin/people/permissions/ROLE`).
- **Add user:** People → Add user. Assign roles on the form.
- **Registration settings:** Configuration → People → Account settings. Choose "Administrators only", "Visitors", or "Visitors, but administrator approval is required".
- **Blocking:** edit a user → Status: Blocked. Cancel account options exist in the same form.

7.3 Node access vs permissions

Permissions are global ("can edit any Event"). **Node access** (grants) adds per-node rules (private content, Content Access/Group modules). Views applies node access automatically (that is why "Disable SQL rewriting" is dangerous).

7.4 Access in code (preview of Chapter 11)

```
requirements:
  _permission: 'access content'     # in routing.yml
requirements:
  _role: 'editor'
requirements:
  _entity_access: 'node.view'
requirements:
  _custom_access: '\Drupal\mymod\Access\Foo::access'
```

Exercise 7.1

Task: Create a role "Editor" that can create/edit own Events and see unpublished content but cannot administer anything. Create user "editor1" with that role and test in a private window.

Answer:

```
ddev drush role:create editor "Editor"
ddev drush role:perm:add editor "access content overview,create event content,edit own
event content,delete own event content,view own unpublished content,access toolbar"
ddev drush user:create editor1 --mail="e1@example.com" --password="Pass123!"
ddev drush user:role:add editor editor1
```

Or in the UI: People → Roles → Add role "Editor" → Permissions grid → tick the Event rows → Save. People → Add user → assign Editor. Open a private window, log in as editor1, visit `/node/add/event` (allowed) and `/admin/structure` (Access denied).

8. Modules, themes, configuration management, recipes

8.1 Install modules and themes

```
ddev composer require drupal/admin_toolbar drupal/gin drupal/gin_toolbar
ddev drush en admin_toolbar admin_toolbar_tools gin_toolbar -y
ddev drush theme:install gin -y
ddev drush config:set system.theme admin gin -y
ddev drush cr
```

UI: **Extend** → tick module → Install. **Appearance** → Install, then "Set as default". Admin theme: `/admin/appearance` bottom "Administration theme".

Core vs contrib vs custom: core ships with Drupal; contrib is on drupal.org (Composer, `drupal/name`); custom is yours.

8.2 Update workflow (know this by heart)

1. Back up: `ddev drush sql:dump > before.sql` or `ddev snapshot`.
2. Check: `ddev composer outdated "drupal/*"`, and **Reports** → **Available updates**.
3. Update: `ddev composer update "drupal/core-*" --with-all-dependencies` (or a single module).
4. `ddev drush updb -y` (database updates).
5. `ddev drush cim -y` if config changed; `ddev drush cr`.
6. Test; commit composer.json, composer.lock, and config.

8.3 Configuration management (CMI)

Meaning: A system that stores site configuration in the database but allows export/import as YAML files so changes move from development → staging → production under version control.

Setup

In `web/sites/default/settings.php`:

```
$settings['config_sync_directory'] = '../config/sync';
```

Workflow

```
ddev drush cex -y      # export DB config to config/sync/*.yml
git add config/ && git commit -m "Add Event content type"
# on another environment
git pull && ddev composer install
ddev drush updb -y
ddev drush cim -y      # import YAML into DB
ddev drush cr
ddev drush config:status   # show differences
```

Related commands

Command	Use
`drush config:get system.site`	Read one config object
`drush config:set system.site name "My Site" -y`	Change one value
`drush config:status`	List differences between DB and files
`drush config:delete name`	Delete a config object
`drush config:export --diff`	Preview before export

Key facts for interviews

- Config files live in `config/sync`; the site **UUID** must match between environments or import fails. Set once: `drush config:set system.site uuid XXXX` (only for a fresh site cloned from the same codebase).
- **Config Split** (contrib) keeps environment-specific config (dev modules like devel, stage_file_proxy) separate.
- **Config Ignore** (contrib) excludes config you do not want overwritten.
- **Config overrides** in `settings.php` via `$config['system.site']['name'] = 'X';` never appear in the UI or export.
- **Active config** = in DB; **sync directory** = YAML files. Config is not content, so nodes, users, terms are not exported.
- **Default config** in modules sits in `config/install` (imported on module install) and `config/optional` (imported if dependencies exist). `config/schema/*.schema.yml` describes the structure of config.

8.4 Recipes (new in Drupal 10.3+, standard in 11)

Meaning: A recipe is a reusable, composable package that applies config, installs modules, and can add content. It is applied once; it is not an ongoing dependency like a module.

```
ddev composer require drupal/drupal_cms_blog     # example recipe package
ddev drush recipe ../recipes/drupal_cms_blog     # apply via Drush
# or without Drush
ddev exec php core/scripts/drupal recipe ../recipes/my_recipe
```

A recipe has `recipe.yml` with `name`, `type`, `recipes:`, `install:`, and `config:` sections (`config: import:` and `actions:`). Recipes live in the `recipes/` folder you already have.

> **Recipe vs install profile vs module:** a profile is chosen once at install and lives for the site's lifetime; a module is ongoing code; a recipe runs once to configure the site.

8.5 Project template files

- `*.info.yml` describes a module/theme (name, type, core_version_requirement, dependencies).
- `*.libraries.yml` declares CSS/JS bundles (asset libraries).
- `*.routing.yml`, `*.services.yml`, `*.permissions.yml`, `*.links.menu.yml`, `*.links.task.yml`, `*.links.action.yml`, `*.module`, `*.install`.

Exercise 8.1

> **Task:** Export your Event content type to code, then delete the content type in the UI, and restore it by importing.

Answer:

```
# 1 set path in settings.php: $settings['config_sync_directory'] = '../config/sync';
ddev drush cex -y
ls config/sync | grep event          # field.storage.node.field_*, node.type.event, ...
# 2 delete content type in UI (Structure > Content types > Event > Delete) - first
delete event nodes
# 3 restore
ddev drush cim -y
ddev drush cr
```

The content type and its fields reappear, but the deleted event nodes do not (content is not config).

9. Popular contributed modules and site-building tools

Interviewers ask "which modules do you use and why?". Learn the purpose of each, and how to install: `ddev composer require drupal/NAME && ddev drush en NAME -y`.

Module	Purpose / when to use
Pathauto (+ Token)	Auto-generate URL aliases from patterns. Use on every content site for SEO-friendly URLs.
Admin Toolbar	Dropdown admin menu for faster navigation.
Gin (+ Gin Toolbar)	Modern admin theme.
Paragraphs	Flexible page building: editors stack reusable components (text, image, slider) inside a field.
Webform	Forms, surveys, submissions, emails, exports. Use for contact/registration forms.
Metatag	Meta tags and Open Graph for SEO and social sharing.
Redirect	301 redirects and automatic redirect when an alias changes.
Simple XML Sitemap	Generates sitemap.xml.
Devel	Debug helpers, generate dummy content (`drush devel-generate:content 20`). Dev environments only.
Views Bulk Operations (VBO)	Bulk actions on view rows.
Config Split / Config Ignore	Environment-specific configuration.
Search API (+ Solr/Database backend)	Full-text search with facets.
Facets	Faceted filtering on search results.
Token	Placeholders like `[node:title]` used by Pathauto, Metatag, Webform.
Entity Reference Revisions	Needed by Paragraphs.
Group / **Content Access**	Per-group or per-node access control.
Easy Breadcrumb	Better breadcrumbs.
Honeypot / reCAPTCHA / Antibot	Spam protection for forms.
Upgrade Status	Checks readiness for the next major Drupal version.
Key / Encrypt	Safe storage of secrets.
Scheduler	Schedule publish/unpublish.
Redis / Memcache	External cache backends.

Stage File Proxy	Load missing files from production on local sites.
Automatic Updates / Project Browser	Browse and install modules from the UI (Project Browser) and automated secure core updates.

Key core modules to know

Core module	Meaning
Views, Views UI	Lists and reports
Field, Field UI	Fields and their admin UI
Layout Builder, Layout Discovery	Drag-and-drop layouts
Media, Media Library	Reusable media
Content Moderation, Workflows	Editorial states
Language, Content Translation, Interface Translation, Configuration Translation	Multilingual
JSON:API, REST, Serialization, Basic Auth	Web services and headless
Migrate, Migrate Drupal	Data import
Navigation	New left sidebar admin menu
Help, Database Logging, Syslog	Docs and logs
BigPipe, Dynamic Page Cache, Internal Page Cache	Performance
Update Manager	Reports available updates
CKEditor 5	Rich text editor
Config, Config Translation	CMI UI
Contextual, Toolbar, Shortcut	Editor conveniences
Single Directory Components (SDC)	Component-based theming
Workspaces	Stage content changes in an isolated workspace
Datetime Range, Responsive Image, Block Content, Taxonomy, Path, Path Alias, Search, Statistics, Tour, History	Misc. site building

Exercise 9.1

> **Task:** Install Admin Toolbar, Paragraphs, Metatag and Webform. Create a contact form with name, email, message and an email to the admin.

Answer:

```
ddev composer require drupal/admin_toolbar drupal/paragraphs drupal/metatag
drupal/webform
ddev drush en admin_toolbar admin_toolbar_tools paragraphs metatag webform webform_ui -y
ddev drush cr
```

Navigate: Structure → Webforms → Add webform (`/admin/structure/webform/add`) → title "Contact" → Build tab → Add element: Name (Text field), Email, Message (Text area) → Settings → Emails/Handlers → Add handler "Email" → To: admin email. Open `/form/contact` and submit. View submissions in the Results tab; check email at `ddev mailpit` .

10. Theming with Twig

10.1 Meaning

A **theme** controls presentation: templates (Twig), styles (CSS), scripts (JS), and regions. Core themes in Drupal 11: **Olivero** (default front end), **Claro** (admin), **Stark** (bare, for learning), **Starterkit** (generator for new themes), **Stable9** (stable markup base).

> **Interview one-liner:** "Themes use the Twig template engine. A theme declares regions and asset libraries in YAML, overrides core markup by copying a template to the theme, and uses preprocess functions to prepare variables."

10.2 Create a custom theme from Starterkit (the Drupal 11 way)

```
ddev exec php core/scripts/drupal generate-theme my_theme --path themes/custom
# result: web/themes/custom/my_theme/
ddev drush theme:install my_theme -y
ddev drush config:set system.theme default my_theme -y
ddev drush cr
```

Starterkit copies a clean theme that does not depend on Olivero afterwards (it is "forked", no base-theme inheritance).

10.3 Theme file anatomy

```
web/themes/custom/my_theme/
  my_theme.info.yml        # name, type, base theme, regions, libraries
  my_theme.libraries.yml   # asset libraries (css/js)
  my_theme.theme           # preprocess functions (PHP)
  css/style.css
  js/app.js
  templates/               # twig overrides
    node--event--teaser.html.twig
    page.html.twig
  components/              # single directory components (SDC)
  logo.svg, screenshot.png
```

my_theme.info.yml

```
name: My Theme
type: theme
description: Custom theme for training
package: Custom
core_version_requirement: ^11
base theme: stable9        # or false to build everything yourself
libraries:
  - my_theme/global
regions:
  header: Header
  primary_menu: Primary menu
  content: Content
  sidebar: Sidebar
  footer: Footer
```

my_theme.libraries.yml

```
global:
  css:
    theme:
      css/style.css: {}
  js:
    js/app.js: {}
  dependencies:
    - core/drupal
    - core/once
```

Attach a library: globally via `libraries:` in info.yml; in Twig: `{{ attach_library('my_theme/global') }}`; in PHP render array: `$build['#attached']['library'][] = 'my_theme/global';`.

10.4 Twig syntax (memorise)

Syntax	Meaning / example
`{{ var }}`	Print a variable (auto-escaped, safe).
`{% if x %} ... {% endif %}`	Logic; also `for`, `set`, `include`, `extends`, `block`.
`{# comment #}`	Comment.
`{{ content.field_location }}`	Render a field with its formatter.
`{{ content|without('field_image') }}`	Render everything except a field.
`{{ label }}`	Node title.
`{{ url }}`	Node URL.
`{{ node.field_location.value }}`	Raw value of a field.
`{{ 'Hello'|t }}`	Translate a string.
`{{ path('entity.node.canonical', {'node': 1}) }}`	URL from route name.

`{{ attributes.addClass('card') }}`	Add a CSS class.
`{{ dump(var) }}`	Debug (with Twig debug on).
`|raw`	Output unescaped HTML. Avoid; XSS risk.
`{% include '@my_theme/partials/card.html.twig' with {title: label} %}`	Include a partial.

10.5 Template suggestions and debugging

Drupal picks the most specific template name. Order for a teaser Event node:

```
node--event--teaser.html.twig
node--event.html.twig
node--teaser.html.twig
node.html.twig
```

Naming rules: double dash `--` separates specificity levels; underscores in machine names turn into hyphens in file names.

Turn on Twig debug (dev only)

```
cp web/sites/default/default.services.yml web/sites/default/services.yml
# edit services.yml under twig.config:
#   debug: true
#   auto_reload: true
#   cache: false
ddev drush cr
```

View page source: HTML comments list "THEME DEBUG", "FILE NAME SUGGESTIONS" (the `x` marks the one in use), and "BEGIN OUTPUT from ...". Copy that core template into your theme's `templates/` and edit it.

> Disable these settings on production. Twig cache off makes the site slow.

10.6 Preprocess functions

```
// my_theme.theme
function my_theme_preprocess_node(array &$variables) {
  $node = $variables['node'];
  if ($node->bundle() === 'event') {
    $variables['is_featured'] = (bool) $node->get('field_featured')->value;
  }
}
function my_theme_preprocess_page(array &$variables) {
  $variables['year'] = date('Y');
}
function my_theme_theme_suggestions_page_alter(array &$suggestions, array $variables) {
  if ($node = \Drupal::routeMatch()->getParameter('node')) {
    $suggestions[] = 'page__' . $node->bundle();
  }
}
```

Hook naming: `THEMENAME_preprocess_HOOK`. Clear cache after adding a new function (hook discovery is cached).

10.7 Single Directory Components (SDC)

Meaning: A component bundles its Twig, CSS, JS, schema, and metadata in one folder. Stable since Drupal 10.3. Reusable and documented.

```
themes/custom/my_theme/components/card/
  card.component.yml
  card.twig
  card.css
  card.js        (optional)
```

```
# card.component.yml
$schema:
https://git.drupalcode.org/project/drupal/-/raw/HEAD/core/assets/schemas/v1/metadata.schema.j
son
name: Card
status: stable
props:
  type: object
  properties:
    title: { type: string, title: Title }
slots:
  body: { title: Body }
```

```
{# card.twig #}
<article {{ attributes.addClass('card') }}>
  <h3>{{ title }}</h3>
  {% block body %}{{ body }}{% endblock %}
</article>
```

Use it in any template: `{{ include('my_theme:card', {title: label}) }}` or `{% embed 'my_theme:card' with {title: label} %}{% block body %}...{% endblock %}{% endembed %}`. CSS and JS are loaded automatically when the component is rendered.

10.8 Responsive images, breakpoints

- `my_theme.breakpoints.yml` defines breakpoints; the Responsive Image module maps them to image styles (`/admin/config/media/responsive-image-style`) to produce `srcset` .
- Lazy loading is enabled by default on images in core formatters (`loading="lazy"`).

10.9 Sub-theme vs Starterkit vs base theme

- **Sub-theme**: inherits templates/CSS from a base theme via `base theme:` . Updates to the base flow in.
- **Starterkit**: a one-time generated copy. No inheritance; you own all the code.
- **Admin theme**: used for admin pages; set at Appearance → Administration theme.

Exercise 10.1

> **Task:** Generate a theme "my_theme", make it default, add a footer with the current year, and override the Event teaser template to show title, date and location in a styled card (use the CSS file).

Answer:

```
ddev exec php core/scripts/drupal generate-theme my_theme --path themes/custom
ddev drush theme:install my_theme -y && ddev drush config:set system.theme default
my_theme -y
```

Create `templates/node--event--teaser.html.twig` :

```
{{ attach_library('my_theme/global') }}
<article{{ attributes.addClass('event-card') }}>
  <h2><a href="{{ url }}">{{ label }}</a></h2>
  <div class="event-card__date">{{ content.field_event_date }}</div>
  <div class="event-card__place">{{ content.field_location }}</div>
</article>
```

Add to `css/style.css` : `.event-card{border:1px solid #ddd;border-radius:8px;padding:1rem;margin-bottom:1rem}` . For the footer year, add the preprocess function from 10.6 and print `{{ year }}` in `page.html.twig` . Run `ddev drush cr` and view `/events` (the view must use "Content: Teaser" row style).

11. Custom module development

11.1 Prerequisites you must know

- **PHP OOP:** classes, namespaces (`namespace Drupal\my_module\Controller;`), interfaces, inheritance, traits, constructor promotion, type declarations, attributes.
- **PSR-4 autoloading:** `Drupal\my_module\Foo` maps to `web/modules/custom/my_module/src/Foo.php` .
- **YAML** for configuration files.

11.2 Create the module

```
ddev drush generate module     # interactive scaffold (Drush Generators)
# or manually:
mkdir -p web/modules/custom/training/src/Controller
```

`training.info.yml` :

```
name: Training
type: module
description: Training examples.
package: Custom
core_version_requirement: ^11
dependencies:
  - drupal:node
```

```
ddev drush en training -y
```

11.3 Routing + Controller (a page)

`training.routing.yml` :

```
training.hello:
  path: '/hello/{name}'
  defaults:
    _controller: '\Drupal\training\Controller\HelloController::greet'
    _title: 'Hello page'
    name: 'World'
  requirements:
    _permission: 'access content'
```

`src/Controller/HelloController.php` :

```php
<?php
namespace Drupal\training\Controller;

use Drupal\Core\Controller\ControllerBase;

class HelloController extends ControllerBase {
  public function greet(string $name): array {
    return [
      '#markup' => $this->t('Hello, @name!', ['@name' => $name]),
    ];
  }
}
```

Visit `/hello/Sam` after `ddev drush cr`. Route parts: **path**, **defaults** (what runs), **requirements** (access), **options** (e.g. `_admin_route: TRUE`, parameter converters). `{name}` is a route parameter passed to the method by name.

Route access requirement options

```yaml
_permission: 'administer nodes'
_role: 'editor'
_user_is_logged_in: 'TRUE'
_access: 'TRUE'                       # public
_entity_access: 'node.update'         # with {node} param
_custom_access: '\Drupal\training\Access\Check::access'
```

11.4 Menu link, local task, action

```yaml
# training.links.menu.yml
training.hello_link:
  title: 'Hello'
  route_name: training.hello
  menu_name: main
  weight: 10

# training.links.task.yml  (tabs)
# training.links.action.yml (buttons like "+ Add")
```

11.5 Permissions

```yaml
# training.permissions.yml
administer training settings:
  title: 'Administer training settings'
  description: 'Change training module settings.'
  restrict access: true
```

11.6 A custom Block plugin

```php
<?php
namespace Drupal\training\Plugin\Block;

use Drupal\Core\Block\Attribute\Block;
use Drupal\Core\Block\BlockBase;
use Drupal\Core\StringTranslation\TranslatableMarkup;

#[Block(
  id: 'training_latest_events',
  admin_label: new TranslatableMarkup('Latest events'),
  category: new TranslatableMarkup('Training'),
)]
class LatestEventsBlock extends BlockBase {
  public function build(): array {
    return [
      '#markup' => $this->t('Latest events go here'),
      '#cache' => ['max-age' => 3600],
    ];
  }
}
```

Place via Structure → Block layout → Place block → category Training. Older code uses docblock `@Block(...)` annotations; Drupal 11 prefers PHP attributes.

11.7 Form API

Form class extends `FormBase` (general) or `ConfigFormBase` (settings stored in config).

```php
<?php
namespace Drupal\training\Form;

use Drupal\Core\Form\FormBase;
use Drupal\Core\Form\FormStateInterface;

class FeedbackForm extends FormBase {
  public function getFormId(): string { return 'training_feedback'; }

  public function buildForm(array $form, FormStateInterface $form_state): array {
    $form['name'] = ['#type' => 'textfield', '#title' => $this->t('Name'), '#required' =>
TRUE];
    $form['email'] = ['#type' => 'email', '#title' => $this->t('Email')];
    $form['msg'] = ['#type' => 'textarea', '#title' => $this->t('Message')];
    $form['actions']['#type'] = 'actions';
    $form['actions']['submit'] = ['#type' => 'submit', '#value' => $this->t('Send')];
    return $form;
  }

  public function validateForm(array &$form, FormStateInterface $form_state): void {
    if (strlen($form_state->getValue('msg')) < 10) {
      $form_state->setErrorByName('msg', $this->t('Message too short.'));
    }
  }

  public function submitForm(array &$form, FormStateInterface $form_state): void {
    $this->messenger()->addStatus($this->t('Thanks, @n!', ['@n' => $form_state-
>getValue('name')]));
    $form_state->setRedirect('<front>');
  }
}
```

```yaml
# routing
training.feedback:
  path: '/feedback'
  defaults:
    _form: '\Drupal\training\Form\FeedbackForm'
    _title: 'Feedback'
  requirements:
    _permission: 'access content'
```

Form lifecycle: build → (user submits) → validate → submit. Common elements: `textfield`, `textarea`, `email`, `number`, `select`, `radios`, `checkboxes`, `checkbox`, `date`, `datetime`, `entity_autocomplete`, `managed_file`, `hidden`, `markup`, `details`, `fieldset`, `container`, `submit`. Security: Drupal adds a CSRF token (`form_token`) automatically.

Config (settings) form

```
class SettingsForm extends ConfigFormBase {
  protected function getEditableConfigNames() { return ['training.settings']; }
  public function getFormId() { return 'training_settings'; }
  public function buildForm(array $form, FormStateInterface $form_state) {
    $config = $this->config('training.settings');
    $form['greeting'] = ['#type' => 'textfield', '#title' => 'Greeting',
      '#default_value' => $config->get('greeting')];
    return parent::buildForm($form, $form_state);
  }
  public function submitForm(array &$form, FormStateInterface $form_state) {
    $this->config('training.settings')->set('greeting', $form_state->getValue('greeting'))-
>save();
    parent::submitForm($form, $form_state);
  }
}
```

Provide defaults in `config/install/training.settings.yml` and a schema in `config/schema/training.schema.yml`.

11.8 Render arrays

```
$build = [
  '#theme' => 'item_list',
  '#items' => ['One', 'Two'],
  '#cache' => ['tags' => ['node_list'], 'contexts' => ['user.permissions'], 'max-age' =>
600],
  '#attached' => ['library' => ['my_theme/global']],
];
$link = ['#type' => 'link', '#title' => 'Go', '#url' =>
\Drupal\Core\Url::fromRoute('<front>')];
$table = ['#type' => 'table', '#header' => ['A','B'], '#rows' => [[1,2],[3,4]]];
```

Properties start with `#`. `#markup` is filtered (XSS-safe); `#plain_text` escapes everything; `#theme` uses a theme hook; `#type` uses a render element.

11.9 Translation and escaping

- `$this->t('Hello @name', ['@name' => $x])` — `@` escapes, `%` escapes and emphasises, `:` is for URLs. **Never** concatenate user input into `t()` strings.
- Use `StringTranslationTrait` in services.

11.10 Install / uninstall and update hooks

```
// training.install
function training_schema() {          // create a custom table
  return ['training_log' => [
    'fields' => ['id' => ['type' => 'serial', 'not null' => TRUE],
                 'msg' => ['type' => 'varchar', 'length' => 255, 'not null' => TRUE]],
    'primary key' => ['id'],
  ]];
}
function training_update_10001() {    // run by drush updb
  // change schema or data here
}
```

Exercise 11.1

Task: Build module "training" with (a) page `/hello/{name}`, (b) a permission-protected page `/admin/training`, (c) a Latest Events block that lists the 3 latest published Event titles, (d) a feedback form.

Answer: Use sections 11.2 - 11.7. For (c) replace the block's `build()` with:

```
public function build(): array {
  $nids = \Drupal::entityQuery('node')
    ->accessCheck(TRUE)
    ->condition('type', 'event')
    ->condition('status', 1)
    ->sort('created', 'DESC')
    ->range(0, 3)
    ->execute();
  $nodes = \Drupal\node\Entity\Node::loadMultiple($nids);
  $items = [];
  foreach ($nodes as $node) { $items[] = $node->toLink(); }
  return [
    '#theme' => 'item_list', '#items' => $items,
    '#cache' => ['tags' => ['node_list:event']],
  ];
}
```

For (b): add a route `training.admin` with path `/admin/training`, `_permission: 'administer training settings'` and `_admin_route: TRUE` in options. Run `ddev drush en training -y && ddev drush cr`.

12. Services, dependency injection, hooks, events, plugins

12.1 Services and the container

Service = a PHP object with one job (logger, mailer, database, current user, entity type manager). The **service container** creates them lazily and shares one instance. Services are declared in `MODULE.services.yml`.

```
# training.services.yml
services:
  training.greeter:
    class: Drupal\training\Greeter
    arguments: ['@current_user', '@logger.factory', '@entity_type.manager']
```

```
namespace Drupal\training;

use Drupal\Core\Session\AccountProxyInterface;
use Drupal\Core\Logger\LoggerChannelFactoryInterface;

class Greeter {
  public function __construct(
    protected AccountProxyInterface $currentUser,
    protected LoggerChannelFactoryInterface $loggerFactory,
  ) {}

  public function greet(): string {
    $this->loggerFactory->get('training')->info('Greeted @u', ['@u' => $this->currentUser-
>getDisplayName()]);
    return 'Hello ' . $this->currentUser->getDisplayName();
  }
}
```

Autowiring is available (`autowire: true`) in Drupal 10.2+. You can also use `\Drupal::service('training.greeter')` in procedural code, but inside classes use **injection**.

Common services

Service id	What
`current_user`	Logged-in user
`entity_type.manager`	Load and query entities
`database`	DB connection
`config.factory`	Read/write config

`logger.factory`	Logging
`messenger`	Status messages
`cache.default`	Cache bin
`state`	Key-value, non-config state
`date.formatter`	Format dates
`request_stack`	Current request
`language_manager`	Languages
`plugin.manager.mail` / `plugin.manager.queue_worker`	Mail and queues
`file_system`, `file_url_generator`	Files
`http_client`	Guzzle HTTP client
`renderer`	Render arrays to HTML
`module_handler`	Invoke hooks, check modules

12.2 Dependency injection in controllers, forms, blocks

Classes that are created by Drupal (controller, plugin, form) use `ContainerInjectionInterface` or `ContainerFactoryPluginInterface` with a static `create()` factory:

```
use Symfony\Component\DependencyInjection\ContainerInterface;
class HelloController extends ControllerBase {
  public function __construct(protected Greeter $greeter) {}
  public static function create(ContainerInterface $container): static {
    return new static($container->get('training.greeter'));
  }
  public function greet(): array { return ['#markup' => $this->greeter->greet()]; }
}
```

Why DI? Testability (swap dependencies with mocks), loose coupling, and no hidden globals. `ControllerBase` already gives helpers like `$this->currentUser()`, `$this->entityTypeManager()`, `$this->config()`.

12.3 Hooks

Meaning: A hook is a named extension point. Modules implement it to alter or add behavior.

Drupal 11 object-oriented hooks (use this style in new code)

```php
namespace Drupal\training\Hook;

use Drupal\Core\Hook\Attribute\Hook;
use Drupal\Core\Entity\EntityInterface;

class TrainingHooks {
  #[Hook('node_presave')]
  public function nodePresave(EntityInterface $entity): void {
    if ($entity->bundle() === 'event' && $entity->isNew()) {
      $entity->setTitle(trim($entity->getTitle()));
    }
  }

  #[Hook('form_alter')]
  public function formAlter(array &$form, \Drupal\Core\Form\FormStateInterface $form_state,
string $form_id): void {
    if ($form_id === 'node_event_form') {
      $form['title']['widget'][0]['value']['#description'] = t('Keep it short.');
    }
  }
}
```

Classic procedural style still works: in `training.module` write `function training_node_presave($node) {...}`. After adding hooks run `drush cr`. Hooks classes sit in `src/Hook/`.

Hooks you must know

Hook	When / use
`hook_form_alter`, `hook_form_FORM_ID_alter`	Change any form
`hook_entity_presave / insert / update / delete` (and `hook_node_*`)	React to entity save/delete
`hook_entity_view_alter`, `hook_node_view`	Alter rendered output
`hook_theme`	Register theme hooks / templates
`hook_preprocess_HOOK`	Prepare Twig variables
`hook_theme_suggestions_HOOK_alter`	Add template suggestions
`hook_menu_links_discovered_alter`, `hook_link_alter`	Alter menus/links
`hook_cron`	Periodic tasks
`hook_install / hook_uninstall / hook_update_N`	Lifecycle
`hook_mail`, `hook_mail_alter`	Email content
`hook_help`	Help text
`hook_query_TAG_alter`	Alter tagged DB queries
`hook_views_data`, `hook_views_pre_render`, `hook_views_query_alter`	Views integration
`hook_node_access`, `hook_entity_access`	Custom access logic
`hook_page_attachments`	Attach libraries/meta to pages

`hook_library_info_alter`	Modify libraries

12.4 Events (Symfony event subscribers)

Meaning: Use events (from Symfony) for request/response lifecycle and for newer subsystems. Hooks vs events: hooks are Drupal-specific and discovered by name; events use a dispatcher and subscribers with priorities.

```
# training.services.yml
  training.redirect_subscriber:
    class: Drupal\training\EventSubscriber\RedirectSubscriber
    tags:
      - { name: event_subscriber }
```

```
namespace Drupal\training\EventSubscriber;
use Symfony\Component\EventDispatcher\EventSubscriberInterface;
use Symfony\Component\HttpKernel\Event\RequestEvent;
use Symfony\Component\HttpKernel\KernelEvents;

class RedirectSubscriber implements EventSubscriberInterface {
  public static function getSubscribedEvents(): array {
    return [KernelEvents::REQUEST => ['onRequest', 30]];   // priority 30
  }
  public function onRequest(RequestEvent $event): void {
    // e.g. redirect /old to /new
  }
}
```

Typical events: `KernelEvents::REQUEST / RESPONSE / EXCEPTION`, `RoutingEvents::ALTER`, `ConfigEvents::SAVE`.

12.5 Plugins

Meaning: A plugin is a class that provides one implementation of a type of functionality. A **plugin type** has a **manager**, **discovery** (attributes / annotations / YAML), and optional **derivers**.

Plugin type	Folder	Example
Block	`src/Plugin/Block`	Latest events
Field type / widget / formatter	`src/Plugin/Field/...`	Custom field
QueueWorker	`src/Plugin/QueueWorker`	Background jobs
Action	`src/Plugin/Action`	Bulk actions
Views field / filter / argument	`src/Plugin/views/...`	Custom view parts
Migrate source/process/destination	`src/Plugin/migrate/...`	Import logic
Condition	`src/Plugin/Condition`	Block visibility
Filter	`src/Plugin/Filter`	Text format filter

Constraint	`src/Plugin/Validation/Constraint`	Entity validation

Annotations vs attributes: annotations (`@Block`) are docblock based and still supported; attributes (`#[Block]`) are the modern PHP 8 way and recommended for new Drupal 11 code.

12.6 Queue example (background work)

```
#[QueueWorker(id: 'training_email', title: new TranslatableMarkup('Send emails'), cron:
['time' => 30])]
class EmailWorker extends QueueWorkerBase {
  public function processItem($data) { /* send mail for $data */ }
}
// add to the queue
\Drupal::queue('training_email')->createItem(['uid' => 5]);
```

Run: `drush queue:list` , `drush queue:run training_email` .

Exercise 12.1

> **Task:** (a) Create a service `training.greeter` . (b) Inject it into a controller. (c) Add a `form_alter` hook that adds a description on the Event node form. (d) Log a message when an Event is saved.

> **Answer:** Follow 12.1 for the service and 12.2 for the controller. For (c) use the `#[Hook('form_alter')]` sample. For (d):
>
> ```
> #[Hook('node_insert')]
> public function nodeInsert(\Drupal\node\NodeInterface $node): void {
>   if ($node->bundle() === 'event') {
>     \Drupal::logger('training')->notice('Event created: @t', ['@t' => $node->label()]);
>   }
> }
> ```
>
> Check `/admin/reports/dblog` or `ddev drush ws --type=training` . The module must be enabled and caches cleared.

13. Entity API, database API, config API, state, cache, queue, cron

13.1 Entity API

Entity types: content entities (node, user, taxonomy_term, media, file, comment, block_content, paragraph) are fieldable and stored in tables; config entities (view, node_type, field_config, role, image_style) are stored in config.

```
use Drupal\node\Entity\Node;

// create
$node = Node::create([
  'type' => 'event',
  'title' => 'Drupal Meetup',
  'field_location' => 'Hyderabad',
  'status' => 1,
]);
$node->save();

// load
$node = Node::load(5);
$nodes = \Drupal::entityTypeManager()->getStorage('node')->loadMultiple([1, 2]);

// read/write fields
$title = $node->label();
$loc = $node->get('field_location')->value;
$node->set('field_location', 'Pune')->save();
$term = $node->get('field_category')->entity;         // referenced entity
$ids = array_column($node->get('field_category')->getValue(), 'target_id');

// delete
$node->delete();

// entity query
$ids = \Drupal::entityQuery('node')
  ->accessCheck(TRUE)
  ->condition('type', 'event')
  ->condition('field_featured', 1)
  ->sort('created', 'DESC')
  ->range(0, 10)
  ->execute();
```

> **Always call** `accessCheck(TRUE|FALSE)` on entity queries (required since Drupal 10). Use TRUE for user-facing output.

Entity concepts for interviews

- **Entity type** (definition class with handlers: storage, view builder, access, list builder, forms, route provider); **bundle**; **fields**: base fields (code-defined, e.g. title) vs configurable fields (UI-defined).
- **Revisions** and **translations** are built into content entities.
- **Custom entity**: define with the `#[ContentEntityType]` attribute, an interface, schema via base field definitions, and handlers. Scaffold with `drush generate entity:content`. Update schema changes with `hook_update_N` and `EntityDefinitionUpdateManager`.
- **Entity access**: `$entity->access('update', $account)`.
- **Typed data / field items**: `$node->field_x` is a `FieldItemList` of `FieldItem`s.

13.2 Database API

```
$database = \Drupal::database();

// select
$result = $database->select('node_field_data', 'n')
  ->fields('n', ['nid', 'title'])
  ->condition('n.type', 'event')
  ->condition('n.status', 1)
  ->orderBy('n.created', 'DESC')
  ->range(0, 5)
  ->execute();
foreach ($result as $row) { echo $row->title; }

// insert / update / delete
$database->insert('training_log')->fields(['msg' => 'Hi'])->execute();
$database->update('training_log')->fields(['msg' => 'Bye'])->condition('id', 1)->execute();
$database->delete('training_log')->condition('id', 1)->execute();

// transaction
$tx = $database->startTransaction();   // rolls back when $tx goes out of scope on exception
```

> **Never** concatenate user input into SQL. Use placeholders (`:name`) or the query builder; this prevents SQL injection.

Entity query vs DB query: use entity query for entity IDs with access/field abstraction; use the DB API for custom tables or performance-critical aggregates.

13.3 Configuration API

```
$config = \Drupal::config('system.site');           // read only
$name = $config->get('name');
$editable = \Drupal::configFactory()->getEditable('training.settings');
$editable->set('greeting', 'Hi')->save();
```

Simple config (a single object like `system.site`) vs **config entities** (many, like views). Schema in `config/schema` is required, or you get "missing schema" errors in tests.

13.4 State API, key/value, settings

- **State**: `\Drupal::state()->set('training.last_run', time());` — environment-specific values not exported (last cron time).
- **Key-value store**: general storage `keyvalue` / `keyvalue.expirable`.
- **Settings** (`settings.php`): `\Drupal\Core\Site\Settings::get('hash_salt')` — credentials and server-specific values.
- **Rule:** editors change it → config; code/environment sets it → settings; runtime-only memory → state; user-generated data → entity.

13.5 Cache API

```
$cid = 'training:feed';
if ($cache = \Drupal::cache()->get($cid)) { $data = $cache->data; }
else {
  $data = expensive();
  \Drupal::cache()->set($cid, $data, time() + 3600, ['node_list']);   // expire + tags
}
\Drupal\Core\Cache\Cache::invalidateTags(['node_list']);
```

Details in Chapter 15.

13.6 Cron

- Cron runs periodic tasks (clear caches, index search, run queues, check updates).
- Run manually: `ddev drush cron` or UI `/admin/config/system/cron`.
- Implement `hook_cron()` (or `#[Hook('cron')]`) or a queue worker with `cron:` time.
- Production: use system cron or a scheduler calling `drush cron` (Automated Cron runs on page requests, not ideal for production).

13.7 Mail

```
$mailManager = \Drupal::service('plugin.manager.mail');
$mailManager->mail('training', 'notify', $to, 'en', ['msg' => 'Hi'], NULL, TRUE);
// hook_mail() builds subject/body
function training_mail($key, &$message, $params) {
  if ($key == 'notify') { $message['subject'] = 'Hello'; $message['body'][] = $params['msg'];
}
}
```

Locally, view mail in `ddev mailpit`.

13.8 Logging and messages

```
\Drupal::logger('training')->error('Failed: @e', ['@e' => $msg]);   // levels: debug info
notice warning error critical alert emergency
\Drupal::messenger()->addStatus('Saved');   // addWarning / addError
```

Exercise 13.1

> **Task:** Using `drush php:eval` create an Event node, then query for featured events and print titles, then change one title.

Answer:

```
ddev drush php:eval '
$n = \Drupal\node\Entity\Node::create(["type"=>"event","title"=>"CLI
Event","field_featured"=>1,"status"=>1]); $n->save();
$ids = \Drupal::entityQuery("node")->accessCheck(FALSE)->condition("type","event")-
>condition("field_featured",1)->execute();
foreach (\Drupal\node\Entity\Node::loadMultiple($ids) as $e) { echo $e->label().PHP_EOL;
}
$n->setTitle("CLI Event v2")->save();'
```

14. Security

14.1 Threats and Drupal's defences

Threat	Meaning	Prevention in Drupal
XSS (cross-site scripting)	Attacker injects JavaScript into pages	Twig auto-escapes; use `#markup` (filtered) or `#plain_text`; `Xss::filterAdmin()`, `Html::escape()`; never `|raw` on user input; restrict text formats.
SQL injection	Attacker alters SQL	Use query builder / placeholders. No string concatenation.
CSRF	Forged requests by a logged-in user	Form tokens automatic; routes with `_csrf_token: 'TRUE'` requirement for links that change data.
Access bypass	Users reach what they should not	Always set route `requirements`, check entity access, use `accessCheck(TRUE)`.
File upload attacks	Malicious files	Restrict extensions, use private files for sensitive data, never allow php/exe.
Brute force	Password guessing	Flood control (core), Login Security, strong passwords, 2FA module.
Session hijacking	Stolen session cookie	HTTPS everywhere, secure cookies, session regeneration (core does).
Information disclosure	Errors leak details	Production: hide errors (`/admin/config/development/logging` → None), keep settings.php out of web writable.
Outdated code	Known vulnerabilities	Apply security updates quickly (Drupal Security Team advisories SA-CORE / SA-CONTRIB).

14.2 Security checklist

- Use HTTPS; set `$settings['trusted_host_patterns']` in settings.php.
- Keep core and contrib updated: `composer audit`, **Available updates** report.
- Least-privilege roles; do not hand out "administer ..." permissions.
- Use "Basic HTML" for authenticated users, "Restricted HTML" for anonymous; Full HTML for admins only.
- Protect `settings.php`: `chmod 444`; put `sites/default/files/private` outside the web root: `$settings['file_private_path'] = '../private';`.
- Disable dev modules (devel, webprofiler) in production.
- Change user 1's name from "admin"; use strong passwords; consider 2FA.
- Block users from PHP execution in files directories (Apache/Nginx rules; Drupal writes .htaccess).
- Check `/admin/reports/status` for warnings, and use the **Security Review** module.
- Do not run `update.php` access publicly (`$settings['update_free_access'] = FALSE;`).

14.3 Code-level security patterns

```
use Drupal\Component\Utility\Html;
use Drupal\Component\Utility\Xss;
$safe = Html::escape($user_input);
$filtered = Xss::filter($user_input);                // allows safe tags
$build = ['#markup' => $this->t('Hi @n', ['@n' => $name])];   // placeholder escapes
// CSRF-protected link route
# routing: requirements: _csrf_token: 'TRUE'
// access result with caching
return AccessResult::allowedIfHasPermission($account, 'administer training')-
>cachePerPermissions();
```

Exercise 14.1

Task: A teammate wrote `'#markup' => '<p>Hello ' . $_GET['name'] . '</p>'`. Explain the vulnerability and fix it.

Answer: It is reflected XSS: `?name=<script>...</script>` runs script in the visitor's browser. Fix: use the Request object, a placeholder and escaping:

```
$name = \Drupal::request()->query->get('name', '');
return ['#markup' => $this->t('Hello @name', ['@name' => $name])];
// or '#plain_text' => $name, and add cache context 'url.query_args:name'
```

15. Performance and caching

15.1 Drupal cache layers

Layer	Meaning
Internal Page Cache	Caches full pages for anonymous users (core module).
Dynamic Page Cache	Caches pages for all users except the personalised parts (placeholders).
BigPipe	Streams cacheable page parts first, then personalised parts.
Render cache	Caches render array fragments.
Twig cache	Compiled templates in PHP files.
Views cache	Query results and rendered output (Tag based / Time based).
Cache bins	Storage buckets: `default, render, data, discovery, bootstrap, config, entity, menu, page, dynamic_page_cache`.
Backend	Database by default; Redis/Memcache for speed.
Browser/CDN cache	Controlled by `Cache-Control` max-age (Performance page) and a CDN or Varnish.

15.2 Cache metadata (the 3 pieces) — very common question

Metadata	Meaning	Example
Tags	"What data did this depend on?" When tagged data changes, the cache is invalidated.	`node:5`, `node_list`, `config:system.site`, `user:3`
Contexts	"What request conditions cause variations?" One cached copy per context value.	`user.roles`, `languages`, `url.path`, `url.query_args`, `route`, `theme`, `user.permissions`
Max-age	"How long can this be cached?" `0` = never cache (avoid; it disables page cache), `Cache::PERMANENT` = until tags invalidate.	`3600`

```
$build['#cache'] = [
  'tags' => ['node_list:event', 'config:training.settings'],
  'contexts' => ['user.roles', 'url.query_args:category'],
  'max-age' => Cache::PERMANENT,
];
// a node: $node->getCacheTags() returns ['node:5']
```

> **Interview one-liner:** "Drupal's cache is correct by default: every render array carries tags, contexts and max-age. Cache invalidation is by tag, so editing a node automatically refreshes every page that displayed it."

Lazy builders / placeholders

For personalised or uncacheable parts (cart count, username) use `#lazy_builder` so the rest of the page stays cached and BigPipe/dynamic page cache fills in the placeholder.

15.3 Performance settings and tuning

- **Configuration → Development → Performance** (`/admin/config/development/performance`): browser and proxy cache max age, aggregate CSS/JS (on in production), clear all caches.
- Enable **Internal Page Cache**, **Dynamic Page Cache**, **BigPipe**.
- Use **Redis** for cache and locks; **Varnish** or a CDN (Cloudflare, Fastly) for full-page caching with tag-based purging.
- Optimise images: image styles, WebP, lazy loading, responsive images.
- Views: avoid loading all entities for huge lists; use fields instead of full entity rendering; add pagers; use caching; avoid heavy relationships.
- Database: indexes on custom tables, avoid N+1 queries (use `loadMultiple`), PHP OPcache enabled, current PHP version.
- Profiling tools: Devel/Webprofiler (dev), Blackfire, New Relic, Xdebug profiler, `ddev xdebug` .

drush cr vs ... when to clear cache

After you change	Do
Twig template (new file), theme hook, library, info.yml, routing, services, hooks, plugin (new class)	`drush cr`
Editing an existing Twig file with Twig cache on	Usually reloads automatically in dev (auto_reload), else `drush cr`
Content or config through the UI	Nothing; tags invalidate automatically
CSS/JS aggregation issues	`drush cr` then hard refresh the browser

Exercise 15.1

> **Task:** A block shows "Latest 3 events" but new events do not appear for an hour. Why, and what is the right fix?

> **Answer:** The block uses `max-age 3600` without a tag for the list. Fix with the right cache tag so it invalidates when any Event changes: `'#cache' => ['tags' => ['node_list:event']]` (and keep `max-age` permanent). Do not set max-age to 0 or call `drush cr` as a workaround. If the content depends on the user's role, add the context `user.roles` .

16. Testing, coding standards, debugging

16.1 Test types

Type	Base class	Meaning / speed
Unit	`UnitTestCase`	One class, no Drupal bootstrap. Fastest.
Kernel	`KernelTestBase`	Partial Drupal bootstrap with selected modules, real DB. Good for services, entities.
Functional	`BrowserTestBase`	Full site, simulated browser (no JavaScript). Slower.
FunctionalJavascript	`WebDriverTestBase`	Real browser with JS (needs Chrome/Selenium).
Nightwatch / Playwright / Cypress	-	End-to-end (front end).

```
// web/modules/custom/training/tests/src/Unit/GreeterTest.php
namespace Drupal\Tests\training\Unit;
use Drupal\Tests\UnitTestCase;
class GreeterTest extends UnitTestCase {
  public function testGreeting(): void { $this->assertSame('Hello', 'Hello'); }
}
```

```
ddev exec "cd web && ../vendor/bin/phpunit -c core modules/custom/training/tests"
# needs phpunit config: copy web/core/phpunit.xml.dist to web/phpunit.xml, set SIMPLETEST_DB
and SIMPLETEST_BASE_URL
```

Testing frameworks: PHPUnit (core), Behat (behaviour), Playwright/Cypress (E2E). Require dev tools with `ddev composer require --dev drupal/core-dev`.

16.2 Coding standards and static analysis

```
ddev composer require --dev drupal/coder phpstan/phpstan mglaman/phpstan-drupal
ddev exec vendor/bin/phpcs --standard=Drupal,DrupalPractice web/modules/custom
ddev exec vendor/bin/phpcbf --standard=Drupal web/modules/custom    # auto-fix
ddev exec vendor/bin/phpstan analyse web/modules/custom --level=5
```

Drupal coding standard: 2-space indentation, `lowerCamelCase` methods/properties, `snake_case` for procedural functions and variables in procedural code (modern code uses camelCase), docblocks for classes and methods, `use` statements at top, files end with a newline.

16.3 Debugging toolbox

Tool	How / when

Watchdog / dblog	`/admin/reports/dblog` or `drush ws`; `drush ws --severity=error --count=20`; tail live: `drush ws --tail`
Error display	`/admin/config/development/logging` → "All messages, with backtrace" (dev only). Or in settings.local.php: `$config['system.logging']['error_level'] = 'verbose';`
Devel	`dpm($var)` / `dd($var)` / `kint($var)` (needs devel + kint). `ddev drush ev "dpm(1)"` alternatives: `\Drupal::logger()`.
Xdebug	`ddev xdebug on`, set breakpoints in VS Code (PHP Debug extension) and listen on port 9003; `ddev xdebug off` when done (it slows PHP).
Twig debug	Section 10.5
Views	Settings → show SQL query and timing; Preview section.
Browser tools	Network, console; Drupal sends headers `X-Drupal-Cache`, `X-Drupal-Dynamic-Cache`, and with debug enabled `X-Drupal-Cache-Tags`.
Status report	`/admin/reports/status`
Local settings	`settings.local.php` with dev overrides, loaded at the bottom of settings.php.

Common problems → fix

Symptom	Likely fix
White screen (WSOD) / "The website encountered an unexpected error"	Check dblog or `ddev logs`; fix PHP error; `drush cr`
Changes to Twig/CSS not showing	`drush cr`; disable CSS aggregation in dev; hard refresh
"Page not found" for a new route	`drush cr`; check indentation in routing.yml
"Module not found" after composer	Check path `web/modules/contrib`; `ddev composer install`; `drush en`
Config import fails with UUID mismatch	Site UUID differs: set same UUID or reinstall from the exported config
"Allowed memory size exhausted"	Increase PHP memory (`.ddev/php/*.ini`); investigate big loops, use batch/queues
Permission denied on files	Fix ownership/chmod on `sites/default/files`
Locked out	`ddev drush uli` or `drush upwd admin newpass`
Update hook pending	`drush updb -y`
Composer conflict	`composer why-not`, `composer prohibits`; update with `--with-all-dependencies`

17. Migrate, JSON:API, headless, multilingual, deployment

17.1 Migrate API

Meaning: A framework to import data from other sources (CSV, JSON, SQL, another Drupal site) into Drupal. Pipeline: **source** → **process** (transform field by field) → **destination**. IDs are tracked in map tables to support rollback and updates.

```
# migrations on a custom module: config/install/migrate_plus.migration.events_csv.yml
# or core-only: migrations/events_csv.yml
id: events_csv
label: Events from CSV
source:
  plugin: csv           # from migrate_source_csv
  path: modules/custom/training/data/events.csv
  ids: [id]
process:
  title: name
  field_location: place
  type:
    plugin: default_value
    default_value: event
destination:
  plugin: entity:node
```

```
ddev composer require drupal/migrate_plus drupal/migrate_tools drupal/migrate_source_csv
ddev drush en migrate_plus migrate_tools migrate_source_csv -y
ddev drush migrate:status
ddev drush migrate:import events_csv
ddev drush migrate:rollback events_csv
ddev drush migrate:import events_csv --update
```

Core Drupal-to-Drupal upgrades (7 → 10/11) use the `migrate_drupal` and `migrate_drupal_ui` modules (`/upgrade`). Common process plugins: `get, default_value, static_map, migration_lookup, concat, explode, callback, format_date, skip_on_empty, sub_process, entity_generate`.

17.2 JSON:API and REST (headless Drupal)

- **JSON:API** (core) automatically exposes all entities: `/jsonapi/node/event`, `/jsonapi/node/event/{uuid}`. Supports `filter`, `sort`, `page[limit]`, `include`, `fields[node--event]=title,field_location`.
- Enable: `ddev drush en jsonapi -y`. Config: `/admin/config/services/jsonapi` (read-only mode recommended unless you need writes).

- Example: `/jsonapi/node/event?filter[field_featured]=1&sort=-created&page[limit]=5&include=field_category`
- **RESTful Web Services** (core): custom resources and Views "REST export" display.
- **GraphQL** (contrib) is an alternative.
- **Authentication:** cookie (same domain), Basic Auth (core), OAuth2 / Simple OAuth (contrib), JWT (contrib). Always use HTTPS.
- **CORS:** in `sites/default/services.yml`: `cors.config: enabled: true, allowedOrigins: ['https://app.example.com']`.
- **Decoupled / headless:** Drupal as the back end, a JS front end (Next.js, Nuxt, React). **Progressively decoupled**: Drupal renders pages and parts use JS components.

17.3 Multilingual

1. Enable `language`, `content_translation`, `locale` (interface translation), `config_translation`.
2. **Add languages:** Configuration → Regional and language → Languages → Add language.
3. **Detection:** Languages → Detection and selection (URL prefix `/fr/...`, domain, session, browser, user).
4. **Enable translation for content type:** Configuration → Regional and language → Content language and translation (`/admin/config/regional/content-language`) → tick Content → Event → mark fields translatable.
5. **Translate:** Node → Translate tab → Add. **Interface strings:** Configuration → Regional and language → User interface translation. **Config:** Configuration → Regional and language → Configuration translation.

Commands: `drush locale:check`, `drush locale:update`. Use `$this->t()` and `{{ 'text'|t }}` for all strings.

17.4 Deployment workflow

```
# local
git pull
ddev composer install
ddev drush updb -y
ddev drush cim -y
ddev drush cr

# production deploy script (typical)
git pull origin main
composer install --no-dev --optimize-autoloader
drush maintenance:set 1   # optional (drush state:set system.maintenance_mode 1)
drush updb -y
drush cim -y
drush cr
drush maintenance:set 0
```

- **Order matters:** code → `composer install` → `updb` → `cim` → `cr`.
- **Environments:** local (DDEV) → dev → staging → production. Content flows production → down (database copy); config flows up (via Git).
- **Hosting:** Acquia, Pantheon, Platform.sh / Upsun, DDEV Pro, Amazon, plain VPS with Nginx + PHP-FPM + MariaDB.

- **CI/CD:** GitHub Actions / GitLab CI / Jenkins: install, lint (phpcs), static analysis (phpstan), tests (phpunit), then deploy.
- **Version control:** commit `composer.json`, `composer.lock`, `config/`, `web/modules/custom`, `web/themes/custom`. Do **not** commit `vendor/` (unless host requires), `web/core` and contrib (installed by composer), `web/sites/default/files`, `settings.local.php`.
- **Backups:** `drush sql:dump --gzip --result-file=../backup.sql` and file backups.
- **Maintenance mode:** `/admin/config/development/maintenance`.

17.5 Multisite, install profiles, distributions

- **Multisite:** one codebase, multiple sites in `web/sites/<name>/`, with `sites.php` mapping domains. Pros: shared code updates; cons: shared failure points.
- **Install profile:** predefines modules/config at install (Standard, Minimal, Demo Umami).
- **Distribution:** profile + code for a use case (Drupal CMS, OpenSocial).

Exercise 17.1

> **Task:** Enable JSON:API and fetch featured events with their category names and only title + location fields.

> **Answer:**
>
> ```
> ddev drush en jsonapi -y
> curl "https://drupal112-training.ddev.site/jsonapi/node/event?
> filter[field_featured]=1&include=field_category&fields[node--
> event]=title,field_location,field_category&fields[taxonomy_term--event_category]=name"
> ```
>
> Anonymous users need "Access content" permission. Terms are returned in the `included` array.

18. Drush and Composer cheat sheet

Prefix with `ddev` locally (`ddev drush cr`). Format: command — when to use.

18.1 Drush

Group	Command	When to use
Basics	`drush status`	Check version, DB, paths
	`drush cr`	Any code/theme/routing/services change; "it doesn't work"
	`drush uli [--uid=2]`	Login link; forgot password
	`drush upwd admin "NewPass1!"`	Reset a password
Modules/themes	`drush en MODULE -y`	Install module
	`drush pmu MODULE -y`	Uninstall module
	`drush pm:list --status=enabled`	See what is installed
	`drush theme:install THEME`	Install theme
	`drush pm:security`	Check Composer packages for security advisories
Config	`drush cex -y`	Export config after building in UI
	`drush cim -y`	Import config after pulling code
	`drush config:status`	Show differences
	`drush cget system.site`	Read config
	`drush cset system.site name "X" -y`	Change config
	`drush config:delete NAME`	Delete a config item
Database	`drush updb -y`	After code update / deploy
	`drush sql:dump > file.sql`	Backup
	`drush sql:cli` / `drush sql:query "SELECT ..."`	Run SQL
Users	`drush user:create NAME --mail=... --password=...`	Create user
	`drush user:role:add ROLE NAME`	Give role
	`drush user:block NAME` / `user:unblock`	Block/unblock
	`drush role:perm:add ROLE "perm"`	Grant permission

Runtime	`drush cron`	Run cron
	`drush ws --count=20` / `--severity=error` / `--tail`	Read logs
	`drush php:eval 'code'` (ev)	Quick PHP
	`drush php:cli` (php)	Interactive shell
	`drush state:set system.maintenance_mode 1`	Maintenance mode
Dev helpers	`drush generate module / controller / form:simple / plugin:block / entity:content / theme / hook`	Scaffold code
	`drush devel-generate:content 20 --bundles=event`	Dummy content
	`drush views:dev` / `views:list`	Views dev mode (disable caches)
	`drush queue:list` / `queue:run NAME`	Queues
	`drush sa @alias`, `drush @prod status`, `drush rsync`, `drush sql:sync @prod @self`	Remote operations via aliases (`drush/sites/*.site.yml`)
Other	`drush deploy`	Runs `updb`, `cr`, `cim`, `cr`, deploy hooks in the right order. Great for pipelines.
	`drush deploy:hook`	Run `hook_deploy_NAME` one-time deploy code
	`drush list` / `drush help CMD`	Discover commands

18.2 Composer

Command	When to use
`composer require drupal/NAME`	Add module/theme
`composer require --dev drupal/devel`	Dev-only package
`composer remove drupal/NAME`	Remove (after uninstalling in Drupal)
`composer update drupal/NAME --with-dependencies`	Update one project
`composer update "drupal/core-*" --with-all-dependencies`	Core update
`composer install --no-dev --optimize-autoloader`	Production deploy
`composer outdated "drupal/*"`	What can be updated
`composer audit`	Security advisories
`composer show drupal/core`	Installed version

`composer why drupal/token` / `why-not`	Dependency reasons/blockers
`composer validate`	Check composer.json
`composer require cweagans/composer-patches`	Apply patches from drupal.org issues (`extra.patches` in composer.json)

18.3 Git (basics for projects)

```
git status
git add -A && git commit -m "Add event type and config"
git checkout -b feature/events
git push -u origin feature/events
git pull --rebase origin main
git log --oneline -10
```

18.4 Quick "when to use what" table

I need to...	**Use**
list content with filters	Views
structure data	Content type + fields
categorise content	Taxonomy
reusable media	Media + Media Library
flexible landing pages	Layout Builder or Paragraphs
forms with emails	Webform
custom logic on content save	Hook or event subscriber
a new page	Route + controller
a reusable output piece	Block plugin
a reusable function	Service
settings page	ConfigFormBase
change markup	Twig override + preprocess
move site settings between environments	Config export/import
move content	Migrate / default content / DB copy
heavy work	Queue + cron
expose data to an app	JSON:API / REST

19. Interview Q&A

Answer in your own words. The short answers below are interview-length.

A. Basics

1. What is Drupal?

An open-source PHP content management framework on Symfony, used for complex, scalable, secure sites with granular permissions and multilingual support.

2. What is the difference between a node, a content type and a field?

A node is one piece of content. A content type is the template/bundle defining its fields. A field is a data slot.

3. What is an entity? A bundle?

An entity is a data object managed by the Entity API (node, user, term, media...). A bundle is a subtype of an entity type (Article is a bundle of node).

4. What is the difference between content and configuration?

Configuration is site structure/settings exportable as YAML (content types, views, roles). Content is user data (nodes, users, terms) kept in the database, moved by migrations or DB copy.

5. What is the difference between core, contrib and custom?

Core ships with Drupal; contrib is community code on drupal.org; custom is code written for your project.

6. What is a block? A region?

A block is a placeable chunk of output. A region is a theme area where blocks go.

7. What is a view mode?

A named display configuration of an entity (Full, Teaser, Card) so the same content renders differently.

8. What is the difference between a widget and a formatter?

Widget = input on the edit form. Formatter = output on display.

9. What is a taxonomy?

A system of vocabularies containing terms used to categorise content.

10. What is Drupal CMS?

A pre-configured product built on Drupal core using recipes, aimed at easy site building; underneath it is the same Drupal.

11. What is Drupal Core's scaffold and recommended-project?

`drupal/recommended-project` is the Composer template for new sites; `drupal/core-composer-scaffold` copies files like index.php into the docroot; `drupal/core-recommended` locks dependency versions.

12. What is the difference between Drupal 10 and 11?

Drupal 11 requires newer PHP (8.3+), updated Symfony 7, removes deprecated APIs, adds OOP hooks, attributes for plugins, Navigation, recipes and SDC maturing, and Single Directory Components.

13. What is a distribution, install profile, recipe?

Profile: initial setup at install. Distribution: profile plus code for a use case. Recipe: a one-time applicable package of config/modules/content.

14. What does `settings.php` contain?

Database credentials, hash salt, config sync directory, trusted hosts, file paths, overrides. Environment-specific; not committed with secrets.

15. What is the front controller?

`web/index.php`, the single entry point that boots Drupal.

B. Site building

16. How do you create a content type and add a field?

Structure → Content types → Add; Manage fields → Add field; configure widget (form display) and formatter (display).

17. How do you show the same content differently in two places?

Create/enable view modes and configure Manage display for each; select the mode in the view or template.

18. What is Views?

A UI query builder for lists, built on displays, fields, filters, sorts, relationships and contextual filters.

19. Contextual filter vs exposed filter?

Contextual filter takes its value from the URL/context. Exposed filter is a form control shown to the visitor.

20. What is a relationship in Views?

A join to a related entity so you can show its fields or filter by them.

21. How do you avoid duplicate rows in a View?

Enable Distinct, use aggregation, or adjust the multi-value field's "Display all values in the same row".

22. How do you add a view to a menu?

Page display → Page settings → Menu → Normal menu entry.

23. How do you display a View in a block?

Add a Block display, then place it in Structure → Block layout.

24. What is the difference between Layout Builder and Paragraphs?

Layout Builder arranges fields/blocks in layouts visually (per content type or per node). Paragraphs lets editors compose content from reusable component types inside a field.

25. Why use Media instead of an Image field?

Media entities are reusable, have metadata (alt, caption), a library UI, and support many sources (video, remote video, documents).

26. How do text formats protect a site?

They run filters (HTML restriction) on user input; weak formats allow XSS, so only trusted roles get Full HTML.

27. How do you create and use roles?

People → Roles; assign permissions on the Permissions page; assign roles to users.

28. What can user 1 do?

Bypasses all permission checks. Use sparingly; use an administrator role for daily work.

29. How do you set up URL aliases?

Manually per node or automatically with Pathauto patterns.

30. How do you enable moderation workflows?

Enable Workflows and Content Moderation; create/edit the Editorial workflow; attach content types; give roles transition permissions.

31. How do you build a contact form?

Core Contact module (`/contact`) for simple needs or Webform for advanced forms.

32. What are image styles?

Reusable image transformations (scale, crop, convert) that create derivative images on demand.

33. How does Drupal handle multilingual?

Language, Content Translation, Interface Translation and Configuration Translation modules provide languages, translatable fields, translated UI strings and translated config.

34. How do you make a field translatable?

Configuration → Content language and translation → enable for the entity bundle and tick the field.

35. What is a revision?

A stored historical version of an entity; you can compare and revert.

C. Config, deployment, tooling

36. How does configuration management work?

Active config lives in the database; `drush cex` writes YAML to the sync directory under Git, and `drush cim` imports it on other environments.

37. What is the deployment order?

`composer install` → `drush updb` → `drush cim` → `drush cr` (or `drush deploy`).

38. What is Config Split?

Contrib module that creates environment-specific config sets (dev modules, debug settings).

39. Why does config import complain about UUID?

The site UUID differs; sites must share one UUID to import each other's config.

40. composer.json vs composer.lock?

json = requirements; lock = exact installed versions. Servers use `composer install` to reproduce.

41. How do you update Drupal core?

Backup; `composer update "drupal/core-*" --with-all-dependencies`; `drush updb`; `drush cr`; test; commit.

42. How do you apply a patch to a contrib module?

Use `cweagans/composer-patches`, add the patch URL in `extra.patches` of composer.json, run `composer update drupal/module`.

43. What is Drush? What is Composer? Difference?

Composer manages PHP packages (installs code). Drush operates on a Drupal site (cache, config, updates).

44. What is DDEV?

A Docker-based local development environment with web server, PHP, database, and tools.

45. How do you migrate a database between environments?

`drush sql:dump` and `drush sql:cli < file`, or `ddev export-db/import-db`, or `drush sql:sync @source @self` with aliases.

46. What is a Drush alias?

A named remote site definition (`@prod`) in `drush/sites/*.site.yml` for running commands remotely.

47. What is the difference between `drush cr` and `drush cc all`?

`cr` rebuilds the registry/container and clears all caches (the right one in modern Drupal); `cc all` is legacy.

48. How do you do a major upgrade 10 → 11?

Update to the latest 10.x, enable Upgrade Status, fix deprecated code and incompatible modules, update PHP, then require core 11 with Composer, run `updb`, test.

49. Which files should and should not be in Git?

In: custom code, config, composer files. Out: vendor/, core/contrib (usually), files directory, local settings, secrets.

50. How do you manage secrets?

Environment variables, `settings.local.php` not in Git, Key module, hosting secret managers.

D. Theming

51. What template engine does Drupal use?

Twig (Symfony); auto-escapes output.

52. How do you override a template?

Enable Twig debug, find the file name suggestion, copy the core template into the theme's `templates/`, rename for specificity, clear cache.

53. What is a preprocess function?

`THEME_preprocess_HOOK(&$variables)` that prepares variables for a template.

54. How do you add CSS/JS?

Define a library in `THEME.libraries.yml`, attach it via info.yml, Twig `attach_library()`, or `#attached`.

55. Base theme vs sub-theme vs Starterkit?

Sub-theme inherits from a base theme; Starterkit generates an independent copy.

56. What is SDC?

Single Directory Components: Twig + CSS + JS + metadata schema in one folder, reusable and documented.

57. What are template suggestions?

Alternative file names Drupal tries, from most specific to least, for a render element.

58. What does `{{ content|without('field_x') }}` do?

Renders remaining content without that field.

59. Why is `|raw` dangerous?

It disables escaping and enables XSS if the value contains user input.

60. How do you debug Twig?

Set `twig.config.debug: true` in services.yml, use `{{ dump(var) }}` or kint, inspect HTML comments.

61. What is the Olivero theme? Claro? Stable9? Starterkit?

Olivero = default front end; Claro = admin; Stable9 = stable markup base; Starterkit = theme generator.

62. What is a render array?

A nested PHP array describing output with `#` properties (`#theme`, `#type`, `#markup`, `#cache`, `#attached`), rendered by the render system.

63. How do you make a theme responsive?

Mobile-first CSS, breakpoints file, responsive image styles, viewport meta (in base templates).

E. Module development

64. What files make a minimum module?

Only `MODULE.info.yml`, in `web/modules/custom/MODULE/`.

65. How do you create a custom page?

Routing YAML entry + controller returning a render array.

66. How do you restrict access to a route?

Requirements: `_permission`, `_role`, `_entity_access`, `_custom_access`.

67. What is a service?

A reusable object defined in services.yml and created by the container.

68. What is dependency injection and why use it?

Passing dependencies via the constructor for testability and loose coupling; avoid `\Drupal::service()` inside classes.

69. Hooks vs events?

Hooks are named Drupal extension points (OOP attribute `#[Hook]` in D11); events use Symfony's dispatcher with subscriber classes and priorities.

70. What is a plugin?

A discoverable class implementing one variant of a feature, e.g. blocks, field formatters, queue workers.

71. Annotation vs attribute plugin discovery?

Both identify plugins; attributes (`#[Block]`) are native PHP 8 and preferred in Drupal 11; annotations still work.

72. How do you create a block programmatically?

A class in `src/Plugin/Block` extending `BlockBase` with `#[Block]` and `build()`.

73. How do you create a form?

Class extending `FormBase` / `ConfigFormBase` implementing `getFormId, buildForm, validateForm, submitForm`, with a route using `_form`.

74. How do you alter a form?

`hook_form_alter` or `hook_form_FORM_ID_alter`.

75. How do you load/save an entity?

`Node::load()`, `Node::create()`, `$node->save()` or the entity type manager storage.

76. Why does entityQuery need `accessCheck()`?

It must be explicit whether to enforce entity access; omitted causes an exception since Drupal 10.

77. How do you write a database query safely?

Database API with placeholders/conditions, never string concatenation.

78. How do you store module settings?

Config API with defaults in `config/install` and a schema, edited with a ConfigFormBase form.

79. State vs config vs settings?

Config = exportable site config; state = environment-specific runtime values; settings = server config in settings.php.

80. What is `hook_update_N`?

One-time database/config update run by `drush updb` when deploying new code.

81. How do you create a custom entity?

Define the entity class with `#[ContentEntityType]`, interface, base fields, handlers and routes (use `drush generate entity:content`), then install it via update hook or module install.

82. What is a queue worker used for?

Deferring heavy or slow tasks (emails, API calls) processed by cron or `drush queue:run`.

83. How do you add a custom field widget or formatter?

Plugin classes in `src/Plugin/Field/FieldWidget` and `FieldFormatter`.

84. How do you create a custom Views field/filter?

Plugin classes under `src/Plugin/views/field` etc. plus `hook_views_data` for custom tables.

85. How do you log errors?

`\Drupal::logger('mod')->error(...)` or an injected logger factory.

86. What is a route parameter converter / upcasting?

Converting `{node}` in a path into a loaded Node object automatically.

87. How do you handle AJAX in forms?

Add `#ajax` with a callback and wrapper to an element; Drupal replaces the wrapper with the callback's render array.

88. How do you send an email?

Mail manager service + `hook_mail()`.

89. How do you run code on cron?

`hook_cron()` or a QueueWorker with a cron time.

90. How do you redirect a user after login?

Use the `user_login_form` alter with a submit handler (`$form_state->setRedirect`), an event subscriber, or the Login Destination module.

F. Caching, performance, security

91. Explain cache tags, contexts and max-age.

Tags invalidate caches when data changes; contexts create variations by request conditions; max-age limits lifetime.

92. What is BigPipe? Dynamic page cache?

BigPipe streams personalised pieces after cached content; dynamic page cache caches pages for authenticated users with placeholders for dynamic parts.

93. What does max-age 0 do?

Makes the response uncacheable; it bubbles up and can disable page caching, so avoid it.

94. How do you speed up a slow Drupal site?

Enable caches and aggregation, Redis, CDN/Varnish, optimise Views and images, use proper cache metadata, profile with Blackfire/Xdebug, upgrade PHP, add DB indexes, reduce modules.

95. How do you prevent XSS?

Rely on Twig auto-escape, placeholders in `t()`, restricted text formats, `Xss::filter`, avoid `|raw`.

96. How do you prevent SQL injection?

Query builder and placeholders.

97. How do you protect against CSRF?

Form tokens; `_csrf_token` route requirement for state-changing links.

98. How do you keep a Drupal site secure?

Update promptly, least privilege, HTTPS, trusted hosts, restricted formats, no debug in prod, private files, backups, monitor advisories.

99. What are Drupal security advisories?

SA-CORE-YYYY-NNN and SA-CONTRIB-YYYY-NNN published by the Drupal Security Team with severity and fix versions.

100. What is the flood control?

Core limits repeated login attempts per user/IP and exposes a Flood API for custom features.

G. Scenario questions

101. "A page shows stale content after an edit." What do you check?

Cache tags on the render array, view cache settings, CDN/Varnish purge, custom code with max-age; not just `drush cr`.

102. "After deployment the site shows config errors."

Check order (updb before cim), UUID mismatch, missing modules in code, `drush config:status`, logs.

103. "Editors need to approve content before publishing."

Content Moderation + Workflows with an Editorial workflow and role-based transitions.

104. "Show related articles by shared tags."

View with contextual filter "Has taxonomy term ID" using the current node's terms (default value: PHP-free option via Views "Taxonomy terms on node" with "Allow multiple values") and exclude the current node via another contextual filter "Content ID" with "Exclude".

105. "Build a content listing with search, filters and pagination."

Views with exposed filters + pager, or Search API with Facets for large datasets.

106. "Import 10,000 rows from CSV."

Migrate API with migrate_plus, migrate_source_csv, run with `drush migrate:import`, rollback to retry.

107. "Expose content to a React app."

JSON:API with filters/includes, CORS, Simple OAuth for authentication, caching with tags.

108. "A module update breaks the site."

Restore the backup/snapshot, check release notes and change records, update in a branch, run tests, fix deprecations, or apply a patch.

109. "Restrict a page to premium members."

Add a role and route/permission, or use Group/Content Access; for node-level, node grants.

110. "Different layout for one landing page."

Layout Builder per-node override, or a template suggestion by node ID/alias.

H. Behavioural answers (prepare 3 stories)

- A bug you fixed (cause → how you found it with logs/Xdebug → the fix → what you learned).
- A feature you built (requirement → Drupal tools chosen → result).
- A mistake or disagreement (what happened → how you handled it).

> **Be honest.** If you built the Event site and the module from this guide, say exactly that. Interviewers value accurate answers with specific details (what you did, which command, what went wrong) more than invented experience.

20. Final revision checklist and mini-project

20.1 The 2-day schedule

Block	Do
Day 1 AM	Chapters 1-4. Start site. Build Event content type. Exercises 1.1, 2.1, 4.1.
Day 1 PM	Chapters 5-7. Build the Views (5.1), taxonomy/menu/block/media (6.1), roles (7.1).
Day 1 night	Chapter 8-9. Config export/import (8.1), install modules (9.1). Read Q&A A-C aloud.
Day 2 AM	Chapter 10-11. Theme + template override (10.1), module with route/block/form (11.1).
Day 2 PM	Chapters 12-15. Service + hook (12.1), entity/DB code (13.1), XSS fix (14.1), cache tags (15.1).
Day 2 night	Chapters 16-18, Q&A D-H. Practise explaining the request lifecycle (1.5) and cache metadata (15.2) without notes.

20.2 Mini-project (portfolio proof)

Build the "Events site" end-to-end and put it on GitHub with a README:

1. Content type Event + vocabulary + media + image style.
2. View `/events` with exposed filter, block with next 3 events, term pages.
3. Roles: Editor, with workflow Draft → Published.
4. Custom theme with Event teaser template and one SDC card component.
5. Custom module `training`: hello page, feedback form, Latest events block, service, `#[Hook]` logging.
6. Config exported to `config/sync` and committed.
7. JSON:API enabled; show a sample request in the README.
8. phpcs passes on the custom module.

20.3 Interview day checklist

- Can I explain: entity/bundle/field, config vs content, request lifecycle, cache tags/contexts/max-age, DI, hooks vs events vs plugins?
- Can I do from memory: create a content type, a View, a role, export config, write a route + controller, write a block, override a template?
- Do I know the update/deploy order and common Drush/Composer commands?
- Do I have 3 real stories from the projects I built?
- Can I say "I don't know that yet, here is how I would find out" (drupal.org docs, api.drupal.org, change records, issue queue)?

20.4 Resources

- api.drupal.org (API reference), drupal.org/docs (official documentation), drupal.org/project/<name> (modules, issue queues), drupal.org/list-changes (change records).
- Drupalize.Me (paid tutorials), Drupal Slack, Stack Exchange (drupal.stackexchange.com), Drupal Easy podcast.
- Certification: Acquia Certified Drupal Site Builder / Developer.

Verify version-specific details (for example attribute names, Starterkit commands, SDC schema URL) against the official documentation for your exact Drupal 11 minor version before the interview; APIs evolve between minor releases.